



NTNU – Trondheim
Norwegian University of
Science and Technology

TET 4565 -Electricity Markets and Energy System Planning, Specialisation Course

Multi-stage Stochastic Programming

Hossein Farahmand

Professor

Autumn 2026

Overview

- Multi-stage optimization (background)
- Mathematical background
- Dual dynamic programming
- Stochastic Dual dynamic programming (SDDP)

Reference

- M. V. F. Pereira and L. M. V. G. Pinto, “Multi stage stochastic optimization applied to energy planning,” *Mathematical Programming*, vol. 52, no. 1-3, pp. 359–375, 1991.
- Pereira, M., Campodonico, N., and Kelman, R. (1999). Application of stochastic dual DP and extensions to hydrothermal scheduling. PSRI Technical Report 012/99.



Dr. Mario Veiga Pereira developed the well-known stochastic dual DP methodology, used by the National System Operator for the dispatch of Brazil’s 95 GW hydrothermal generation systems; and by the Wholesale Energy Market for the calculation of spot prices.

Multi-stage stochastic optimization applied to energy planning

M.V.F. Pereira and L.M.V.G. Pinto

Electric Engineering Department, Catholic University of Rio de Janeiro, P.O. Box 38063, Gavea, 22452 Rio de Janeiro, RJ, Brazil

Received October 1988

Revised manuscript received December 1989

This paper presents a methodology for the solution of multistage stochastic optimization problems, based on the approximation of the expected-cost-to-go functions of stochastic dynamic programming by piecewise linear functions. No state discretization is necessary, and the combinatorial “explosion” with the number of states (the well known “curse of dimensionality” of dynamic programming) is avoided. The piecewise functions are obtained from the dual solutions of the optimization problem at each stage and correspond to Benders cuts in a stochastic, multistage decomposition framework. A case study of optimal stochastic scheduling for a 39-reservoir system is presented and discussed.

1. Introduction

This technical note describes an algorithm for the solution of multistage stochastic optimization problems. The solution approach, called *stochastic dual dynamic programming* (SDDP), is based on the approximation of the expected-cost-to-go functions of stochastic dynamic programming by piecewise linear functions. These approximate functions are obtained from the dual solutions of the optimization problem at each stage and can be interpreted as Benders cuts in a stochastic, multistage decomposition algorithm. No state discretization is necessary, and the combinatorial “explosion” with the number of states (the well known “curse of dimensionality” of dynamic programming) is avoided. The algorithm is also suitable for implementation in parallel processors. The application of the algorithm is illustrated in a case study of optimal stochastic scheduling for a 39-reservoir system.

2. Dual dynamic programming — deterministic case

The concepts of dual dynamic programming will be illustrated with the following linear programming problem:

$$\begin{aligned} \text{Min} \quad & c_1 x_1 + c_2 x_2 \\ \text{subject to} \quad & A_1 x_1 \geq b_1, \\ & E_1 x_1 + A_2 x_2 \geq b_2. \end{aligned} \tag{1}$$

Multi-stage Optimisation

- Canonical multi-stage deterministic problem

$$\min_{x,y} \left\{ \sum_{t=1}^T f_t(\mathbf{x}_t, \mathbf{y}_t) : (\mathbf{x}_{t-1}, \mathbf{x}_t, \mathbf{y}_t) \in X_t \ \forall t \right\}$$

- $\mathbf{x}_t$ = state variables; $\mathbf{y}_t$ = local/stage variables
- Linear objective and constraints

$$\boxed{A_t \mathbf{x}_{t-1} + B \mathbf{x}_t} + C_t \mathbf{y}_t \geq b_t$$

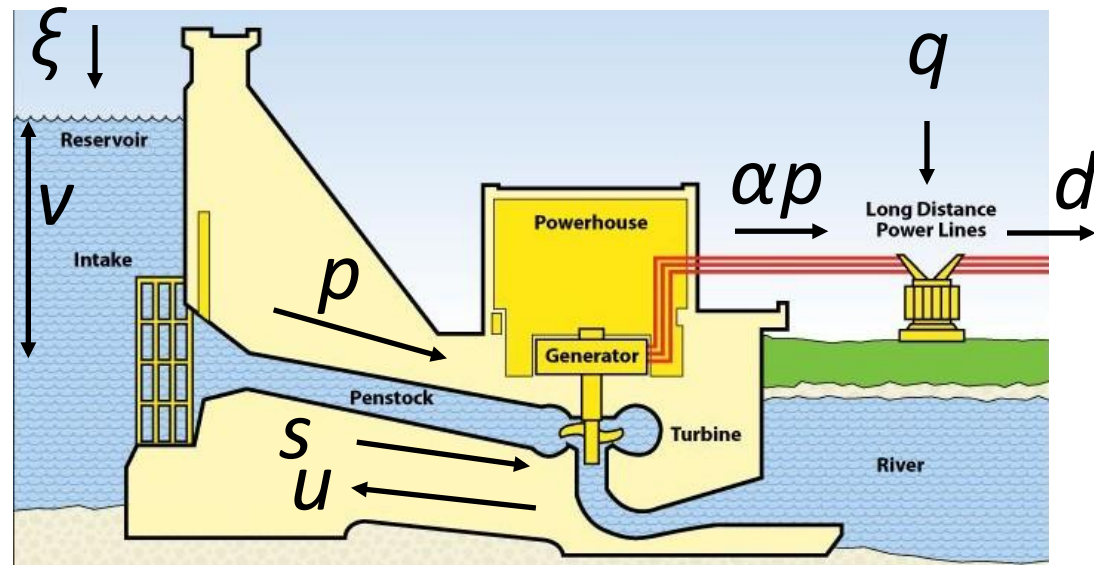
- Bounded and feasible

Example: Hydro Power Planning

How much hydropower is to generate in each period to satisfy demand?

$$x_t = \begin{pmatrix} v_t \end{pmatrix}$$

$$y_t = (p_t, q_t, u_t, s_t)$$



$$\min \sum_t (b_t q_t + c_t u_t + g_t s_t)$$

s.t.

$$v_t = v_{t-1} + \xi_t - p_t + u_t - s_t \quad \forall t$$

$$\alpha p_t + q_t = d_t \quad \forall t$$

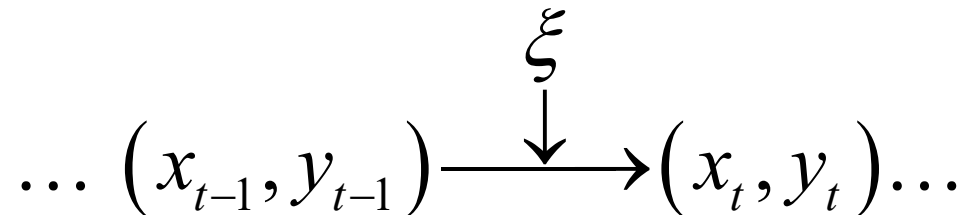
$$0 \leq v_t \leq v^{\max} \quad \forall t$$

$$p_t, q_t, u_t, s_t \geq 0 \quad \forall t$$

Multi-stage Stochastic Optimisation

$$\min_{x,y} \left\{ \sum_{t=1}^T f_t(x_t, y_t) : (x_{t-1}, x_t, y_t) \in X_t \ \forall t \right\}$$

- Stochastic Data: $\left\{ \xi_t = (f_t, X_t) \right\}_{t=1}^T$
- Dynamics: (how decisions and revealed information interact with each other.



Formulation

What happens afterwards (nested formulation)

$$\min_{(x_1, y_1) \in X_1} \left\{ f_1(x_1, y_1) + \mathbf{E} \left[\min_{(x_2, y_2) \in X_2(x_1)} \left\{ f_2(x_2, y_2) + \dots \right. \right. \right. \\ \left. \left. \left. + \mathbf{E} \left[\min_{(x_T, y_T) \in X_T(x_{T-1})} \left\{ f_T(x_T, y_T) \right\} \right] \right] \right] \right\}$$

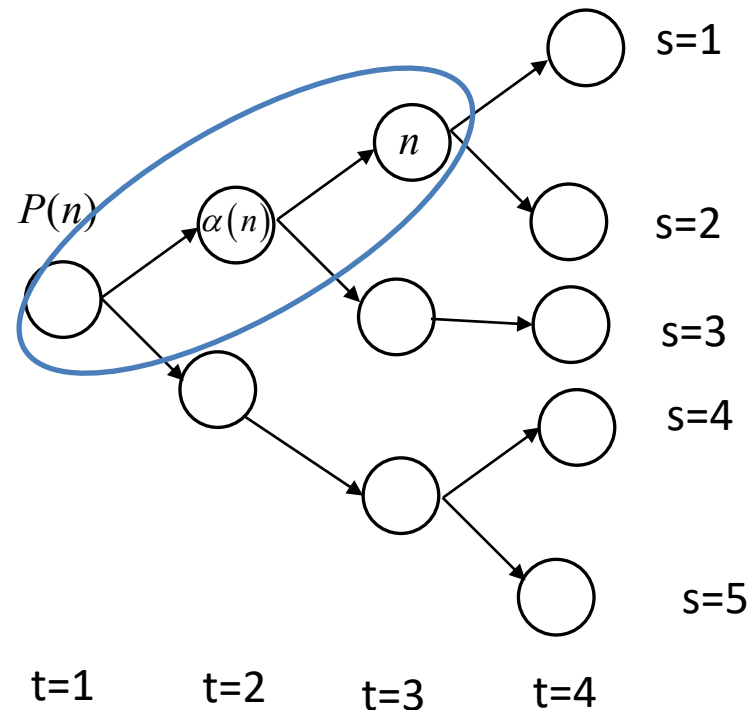
- Optimize expected cost
- Expectation is w.r.t conditional distribution, i.e., at stage t

$$\mathbf{E}_{\xi[t, T]} \left[\cdot \mid \hat{\xi}_{[1, t-1]} \right]$$

- Natural extension of the two-stage formulation
- Decisions in stage t depend on history up to stage t (called a policy)
- **Assume** “complete recourse”: For any value of state variables and data realization, there exists a feasible local/stage solution.

Scenario Tree

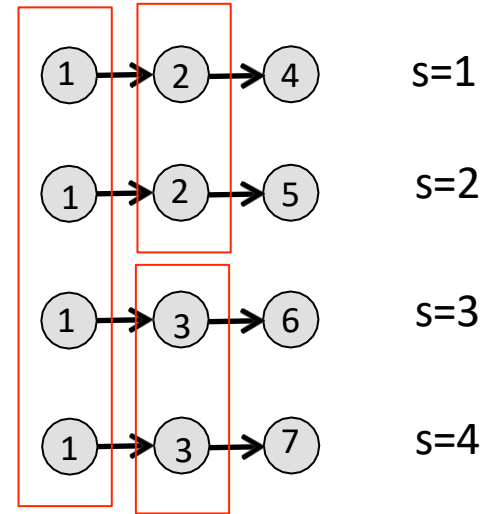
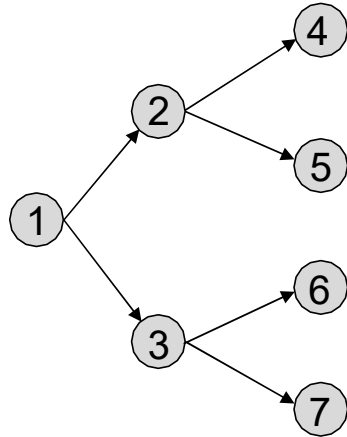
- Dynamic uncertainty modelling: How the information is going to be revealed
- The entire stochastic process is given in the form of a **scenario tree** (implicit Markov Chain)



Formulations

- Extensive form → very huge problem
- Scenario formulation
- Dynamic programming formulation

Scenario Formulation (Scenario Decomposition)



$$\min_{x,y} \sum_s p_s \left(\sum_t f_{s,t} (x_{s,t}, y_{s,t}) \right)$$

s.t.

$$(x_{s,t-1}, x_{s,t}, y_{s,t}) \in X_{s,t}$$

$$\forall s, t$$

$$x_{s,t} = x_{s',t}$$

$$\forall (s, s') \in N_t, \forall t$$

Non-Anticipativity Constraints

Dynamic Programming-Stagewise Decomposition

- Formulation:

$$\min_{x_1, y_1} \left\{ f_1(x_1, y_1) + \alpha_1(x_1) : (x_{a(1)}, x_1, y_1) \in X_1 \right\}$$

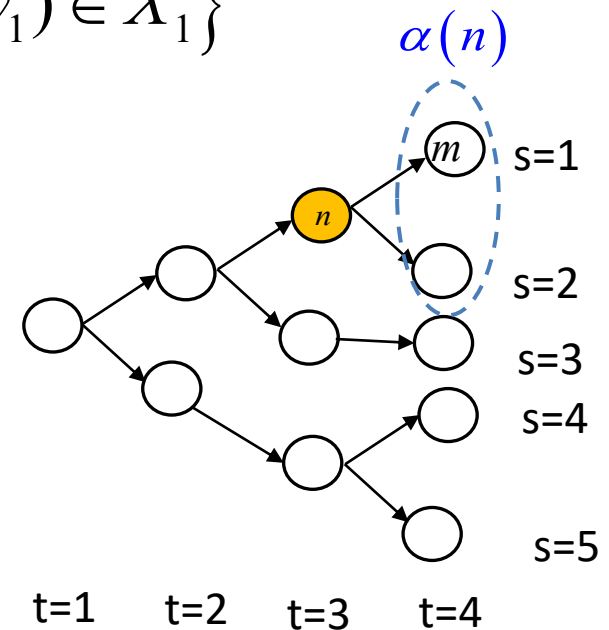
- Expected Cost-to-go (ECTG) function:

$$\alpha_n(x_n) := \sum_{m \in C(n)} q_{nm} Q_m(x_n)$$

Conditional probability

- Cost-to-go function:

$$Q_m(x_n) = \min_{x_m, y_m} \left\{ f_m(x_m, y_m) + \alpha_m(x_m) : (x_n, x_m, y_m) \in X_m \right\}$$



Solution Approaches

Extensive form is usually too large to be solved as one monolithic problem. Need for decomposition approaches.

- Two stage approximation + Rolling horizon
- Scenario Decomposition:
 - Progressive Hedging
- Stagewise Decomposition
 - Stochastic Dual Dynamic Programming
 - Nested Benders

Mathematical background

Mathematical background

Consider the following two-stage deterministic problem:

subject to

$$\min_{x_1, x_2} \overbrace{c_1 x_1}^{\text{First-stage cost}} + \underbrace{c_2 x_2}_{\text{Second-stage cost}}$$
$$A_1 x_1 \geq b_1 \quad \longleftarrow \text{First-stage constraint}$$
$$\underbrace{E_1 x_1 + A_2 x_2}_{\text{Linking constraint}} \geq b_2$$

Let's try to solve the first- and second-stage problems separately!

Mathematical background

Consider the following two-stage deterministic problem:

$$\begin{array}{ll} \min_{x_1, x_2} & c_1 x_1 + c_2 x_2 \\ \text{subject to} & \\ & A_1 x_1 \geq b_1 \\ & E_1 x_1 + A_2 x_2 \geq b_2 \end{array}$$

First-stage problem:

$$\begin{array}{ll} \min_{x_1} & c_1 x_1 + \alpha_1(x_1) \\ \text{subject to} & \\ & A_1 x_1 \geq b_1 \end{array}$$

Second-stage problem:

$$\begin{array}{ll} \alpha_1(x_1) = \min_{x_2} & c_2 x_2 \\ \text{subject to} & \\ & A_2 x_2 \geq b_2 - E_1 x_1 \end{array}$$

$\alpha_1(x_1)$: The second-stage cost as a function of the first-stage x_1 decisions (future cost function or ECTG)

Mathematical background

Consider the following two-stage deterministic problem:

First-stage problem:

$$\begin{aligned} & \min_{x_1} c_1 x_1 + \alpha_1(x_1) \\ & \text{subject to} \\ & A_1 x_1 \geq b_1 \end{aligned}$$

Second-stage problem:

$$\begin{aligned} & \alpha_1(x_1) = \min_{x_2} c_2 x_2 \\ & \text{subject to} \\ & A_2 x_2 \geq b_2 - E_1 x_1 \end{aligned}$$

Note: x_1 appears in the second-stage problem!
So, these two problems cannot be still solved separately!



Mathematical background

Solve the first-stage and the second-stage problems separately

- **Step 1)** Discretize x_1 into a set of trial segment values $\{\hat{x}_{1i}, i = 1, \dots, n\}$
- **Step 2)** Solve the second-stage problem for each trial value
- **Step 3)** Construct cost function $\alpha(x_1)$.
Intermediate values of $\alpha(x_1)$ are obtained by interpolation from the neighboring discretized values.
- **Step 4)** Solve the first-stage problem using the future cost function constructed

First-stage problem:

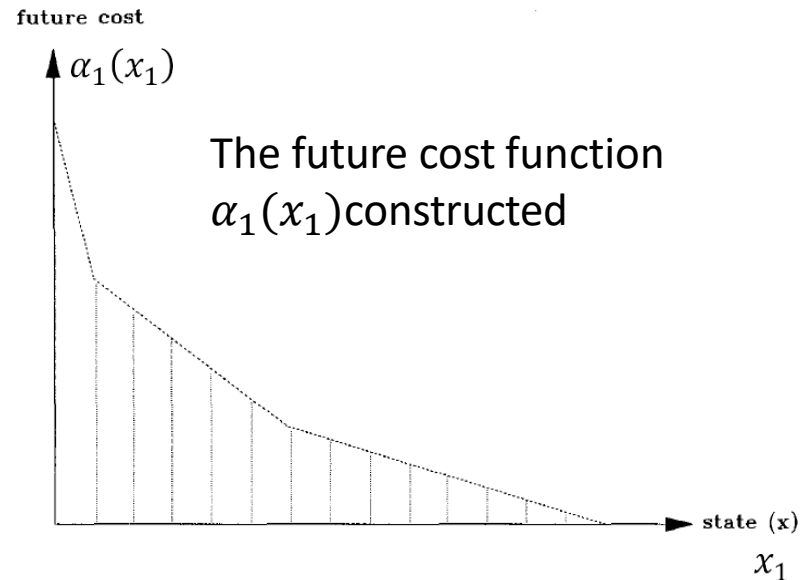
$$\begin{aligned} & \min_{x_1} c_1 x_1 + \alpha_1(x_1) \\ & \text{subject to} \\ & A_1 x_1 \geq b_1 \end{aligned}$$

Second-stage problem:

$$\begin{aligned} & \alpha_1(x_1) = \min_{x_2} c_2 x_2 \\ & \text{subject to} \\ & A_2 x_2 \geq b_2 - E_1 x_1 \end{aligned}$$

Mathematical background

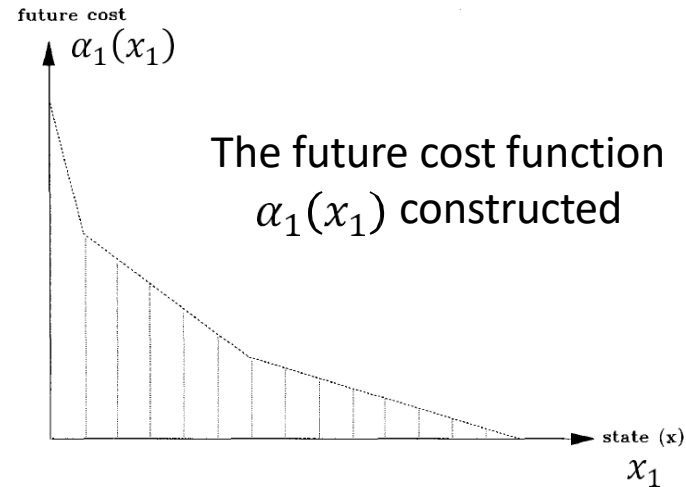
- **Step 1)** Discretize x_1 into a set of trial segment values $\{\hat{x}_{1i}, i = 1, \dots, n\}$
- **Step 2)** Solve the second-stage problem for each trial value
- **Step 3)** Construct cost function $\alpha(x_1)$. Intermediate values of $\alpha(x_1)$ are obtained by interpolation from the neighboring discretized values.
- **Step 4)** Solve the first-stage problem using the future cost function constructed



Similar to

Dynamic programming (DP)

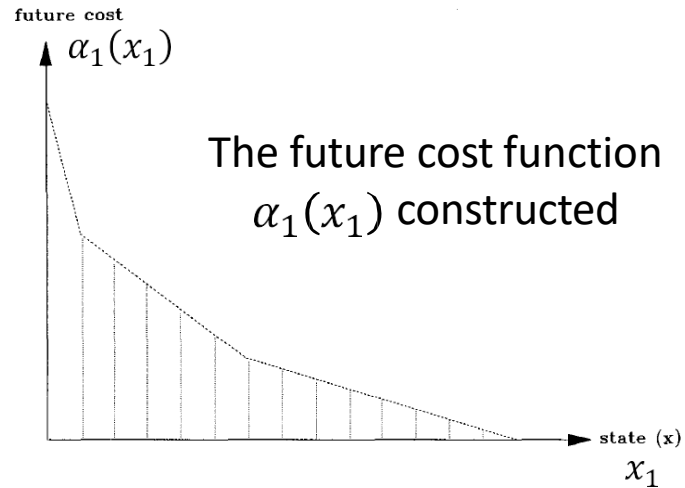
Mathematical background-DP approach



The DP approach has many attractive features

- It can be extended to stochastic cases
- It can be easily extended to multistage problems
- It can accommodate discrete values, nonlinearities

Mathematical background



What is the main drawback of dynamic programming (DP)?

- DP needs to discretize the decision variables x_1 , which results in computational issues!
- For example, 10 decision variables and 4 discretized value for each variable leads to 4^{10} discrete values!

Curse of dimensionality!!!!!! ☹️

Mathematical background

Alternative approach!

Dual dynamic programming (DDP)!

Advantage:

To approximate the future cost function $\alpha_1(x_1)$ by analytical functions rather than a set of discrete values!

Dual dynamic programming (DDP)

Second-stage problem:

$$\begin{aligned} \alpha_1(x_1) &= \min_{x_2} c_2 x_2 \\ \text{subject to} \\ A_2 x_2 &\geq b_2 - E_1 x_1 \quad : \pi \end{aligned}$$

Without loss of generality, we assume variables x_1 and x_2 are non-negative. This could be relaxed.

Dual variable

Dual dynamic programming (DDP)

Second-stage problem:

$$\begin{aligned} \alpha_1(x_1) &= \min_{x_2} c_2 x_2 \\ \text{subject to} \\ A_2 x_2 &\geq b_2 - E_1 x_1 \quad : \pi \end{aligned}$$

Dual of the second-stage problem:

$$\begin{aligned} \max_{\pi} \quad & \pi (b_2 - E_1 x_1) \\ \text{subject to} \\ c_2 &\geq \pi A_2 \end{aligned}$$

In the optimal solution the
strong duality theorem
implies:

$$\alpha_1(x_1) = \pi^* (b_2 - E_1 x_1)$$

Dual dynamic programming (DDP)

Second-stage problem:

$$\begin{aligned} \alpha_1(x_1) &= \min_{x_2} c_2 x_2 \\ \text{subject to} \\ A_2 x_2 &\geq b_2 - E_1 x_1 \quad : \pi \end{aligned}$$

Interpretation: There is a linear relation between x_1 and the future cost function $\alpha_1(x_1)$ if the sensitivity π^* is known! x_2 does not appear

Dual of the second-stage problem:

$$\begin{aligned} \max_{\pi} \quad & \pi (b_2 - E_1 x_1) \\ \text{subject to} \\ & c_2 \geq \pi A_2 \end{aligned}$$

$$\alpha_1(x_1) = \pi^* (b_2 - E_1 x_1)$$

Dual dynamic programming (DDP)

$$\alpha_1(x_1) = \pi^*(b_2 - E_1 x_1)$$

In the **optimal** solution

(**strong duality theorem**: value of primal objective function = value of dual objective function)

$$\alpha_1(x_1) \geq \pi(b_2 - E_1 x_1)$$

In any **feasible** (not necessarily optimal) point within the feasible space

(**weak duality theorem**: value of primal objective function $\geq$ value of dual objective function)

Challenge: we do not know the optimal value for dual variable π^* !

Dual dynamic programming (DDP)

$$\alpha_1(x_1) = \pi^*(b_2 - E_1 x_1)$$

Assume $\pi^1, \pi^2, \dots, \pi^n$ are possible values for π . Let's assume the optimal value for dual variable, i.e., π^* is included in this set of possible values. Then, $\alpha_1(x_1)$ can be characterized

$$\alpha_1(x_1) = \min_{\alpha}$$

subject to

$$\alpha \geq \pi^1(b_2 - E_1 x_1)$$

$$\alpha \geq \pi^2(b_2 - E_1 x_1)$$

$$\vdots$$

$$\alpha \geq \pi^n(b_2 - E_1 x_1)$$

Recall:

- In non-optimal points, the value of α representing the primal objective function $\alpha_1(x_1)$ is greater than the dual objective function (weak duality)!
- The values of those objective functions are equal in the optimal point (strong duality)!

Dual dynamic programming (DDP)

$$\alpha_1(x_1) = \pi^*(b_2 - E_1 x_1)$$

Assume $\pi^1, \pi^2, \dots, \pi^n$ are possible values for π . Let's assume the optimal value for dual variable, i.e., π^* is included in this set of possible values. Then, $\alpha_1(x_1)$ can be characterized

$$\alpha_1(x_1) = \min_{\alpha}$$

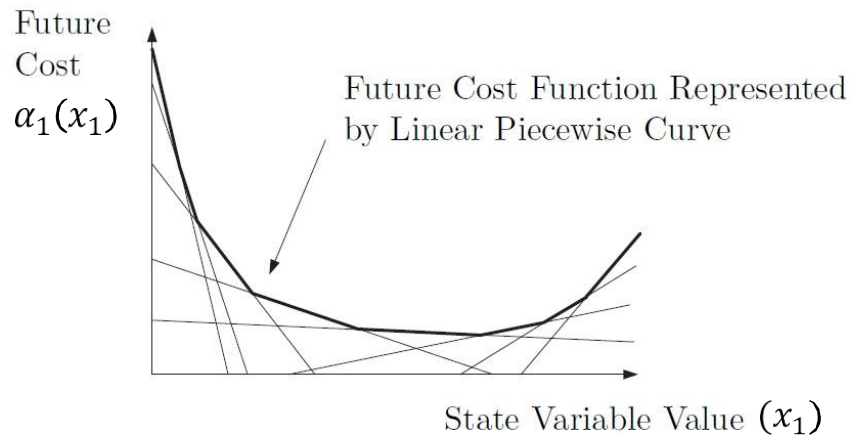
subject to

$$\alpha \geq \pi^1(b_2 - E_1 x_1)$$

$$\alpha \geq \pi^2(b_2 - E_1 x_1)$$

$$\vdots$$

$$\alpha \geq \pi^n(b_2 - E_1 x_1)$$



Note: this means that we can construct a piecewise linear function for $\alpha_1(x_1)$ problem (analytically but approximately) without need to discretize x_1 !

Dual dynamic programming (DDP)

$$\alpha_1(x_1) = \pi^*(b_2 - E_1 x_1)$$

Assume $\pi^1, \pi^2, \dots, \pi^n$ are possible values for π . Let's assume the optimal value for dual variable, i.e., π^* is included in this set of possible values. Then, $\alpha_1(x_1)$ can be characterized

$$\begin{aligned} \alpha_1(x_1) &= \min_{\alpha} \alpha \\ \text{subject to} \\ \alpha &\geq \pi^1(b_2 - E_1 x_1) \\ \alpha &\geq \pi^2(b_2 - E_1 x_1) \\ &\vdots \\ \alpha &\geq \pi^n(b_2 - E_1 x_1) \end{aligned}$$

Recall the first-stage problem:

$$\begin{aligned} \min_{x_1} \quad & c_1 x_1 + \alpha_1(x_1) \\ \text{subject to} \quad & A_1 x_1 \geq b_1 \end{aligned}$$

Let's merge them!

Dual dynamic programming (DDP)

The updated first-stage problem including the piecewise linear function $\alpha_1(x_1)$

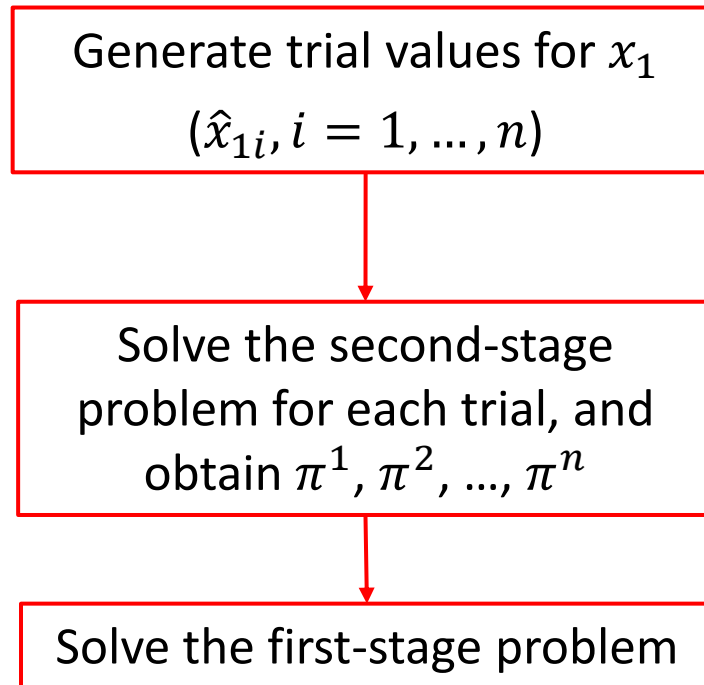
$$\begin{array}{ll}\min_{\alpha, x_1} & c_1 x_1 + \alpha \\ \text{subject to} & \\ & A_1 x_1 \geq b_1 \\ & \alpha \geq \pi^1 (b_2 - E_1 x_1) \\ & \alpha \geq \pi^2 (b_2 - E_1 x_1) \\ & \vdots \\ & \alpha \geq \pi^n (b_2 - E_1 x_1)\end{array}$$

- This means that we could solve the first-stage problem (including variable x_1 only) independent from the second-stage problem (which includes variable x_2)!
- **Assumption:** we have the set of possible values for dual variable π^* !

Dual dynamic programming (DDP)

How to generate possible values for π^* , i.e., $\pi^1, \pi^2, \dots, \pi^n$?

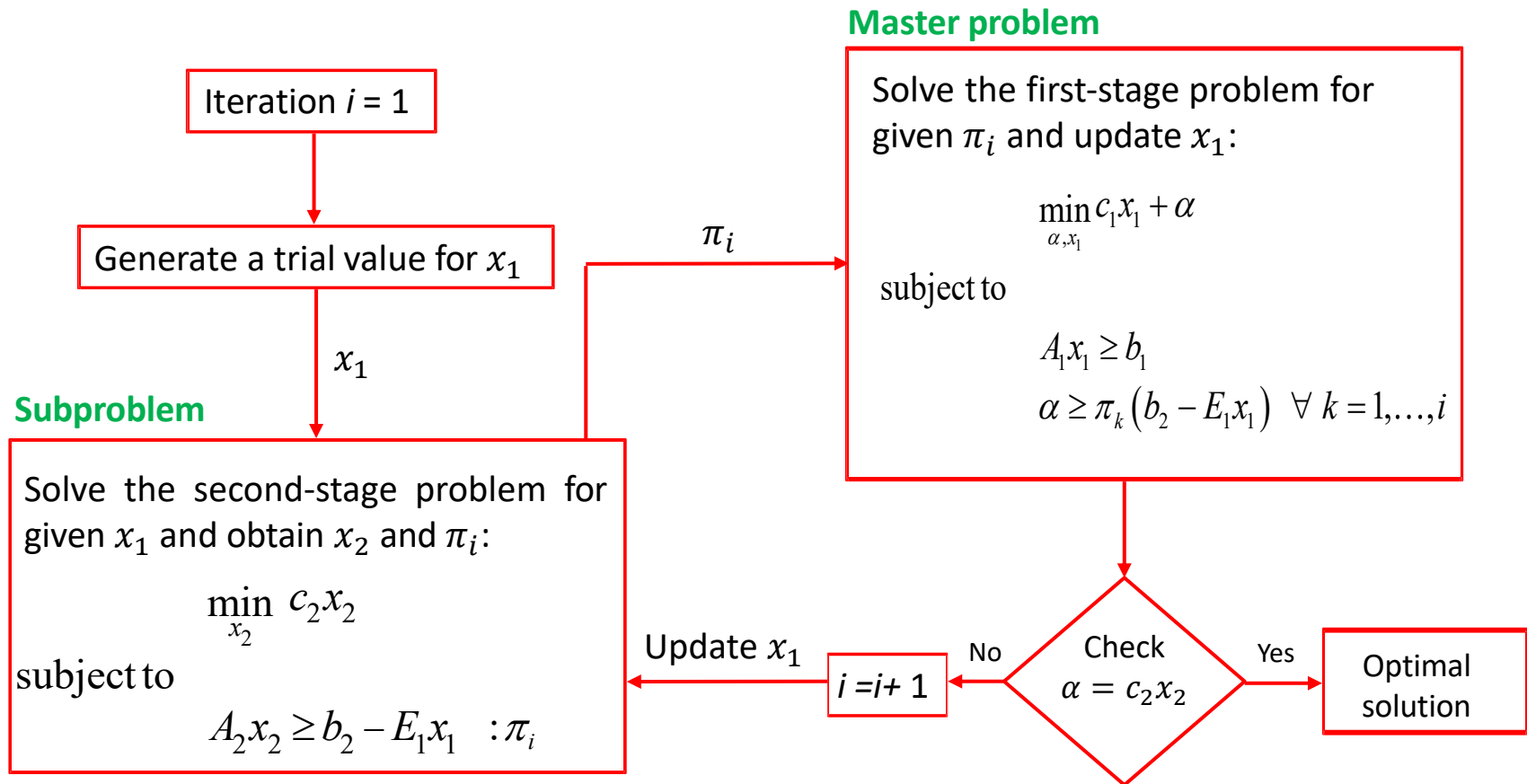
Option 1:



Dual dynamic programming (DDP)

How to generate possible values for π^* , i.e., $\pi^1, \pi^2 \dots, \pi^n$?

Option 2: (a systematic iterative approach)

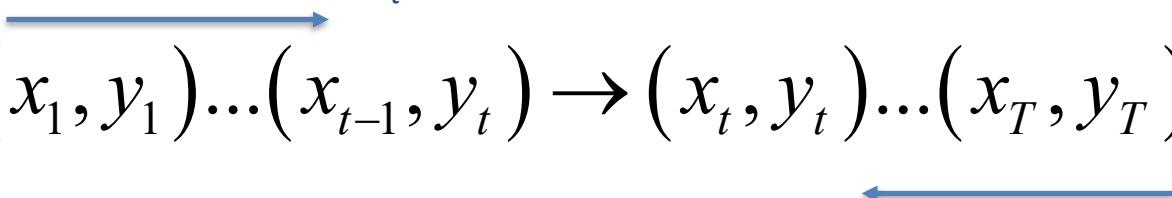


α is a lower band and $c_2 x_2$ is an upper bound for the problem

From Dual Dynamic Programming (DDP) → Multi-Stage DDP

- In Dual Dynamic Programming (DDP), we iteratively solve a two-stage problem:
 - Master problem: chooses x_1 and lower bound α
 - Subproblem: solves for x_2 and generates dual multipliers π_i
 - Iterations refine lower and upper bounds until convergence
- The same logic can be generalized to T stages
(Stage 1 → Stage 2 → ... → Stage T)
Introduces forward simulation and backward recursion
 - Forward simulation ($t=1 \rightarrow T$) calculates x_t at each stage
 - Backward recursion ($t=T \rightarrow 1$) calculates π_t per stage to construct cuts or approximate cost-to-go functions $\hat{\alpha}_t(x_t)$
- Multi-Stage DDP extends the two-stage iterative concept into a recursive, forward–backward structure for solving large-scale, multi-period stochastic problems

Forward simulation: x_t


$$(x_1, y_1) \dots (x_{t-1}, y_{t-1}) \rightarrow (x_t, y_t) \dots (x_T, y_T)$$

Backward recursion: π_t and construct cuts

Multi-stage DDP

- Assume that we have T stages i.e., $t = 1 \dots T$
- **Forward simulation:**
 - α represents the expected cost-to-go function at each stage t
 - At each stage the problem can be formulated as below:

$$z = \min_{x_t, \hat{\alpha}} c_t x_t + \hat{\alpha}_t(x_t)$$

subject to

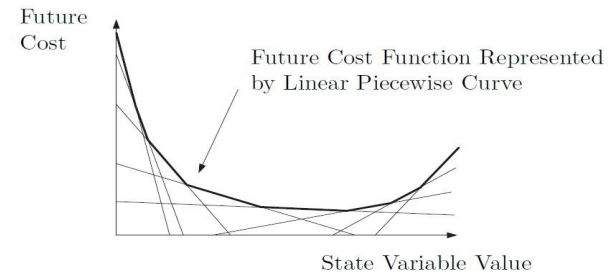
$$A_t x_t \geq b_t$$

- **Backward recursion:**
 - An approximation to future cost can be constructed *as a hyperplane*:

$$\hat{\alpha}_t(x_t) = \min_{\pi} \alpha$$

subject to

$$\alpha \geq \pi^i (b_t - E_t x_{t-1}) \quad i = 1 \dots n$$



- When we apply the iterative approach
 - The lower bound z is $\underline{z} = c_t \hat{x}_t + \hat{\alpha}_t$
 - The upper bound for z is $\bar{z} = c_t \hat{x}_t + \alpha_t(\hat{x}_t)$

Multi-stage dual dynamic programming (DDP)

Algorithm

Step (a). Let T be the planning horizon; initialize $\hat{\alpha}_t(x_t) = 0$ for $t = 1, \dots, T$; $\bar{z} = \infty$.

Step (b). Solve the approximate first stage problem (10); let $\hat{x}_1$ be the optimal solution.

Step (c). Calculate the lower bound $\underline{z}$ as in (12); if $\bar{z} - \underline{z} \leq \varepsilon$, stop; otherwise, go to Step (d).

Step (d). Repeat for $t = 2, \dots, T$ (*forward simulation*).

Solve the optimization problem for stage t , trial decision $\hat{x}_{t-1}$:

$$\begin{aligned} \text{Min} \quad & c_t x_t + \hat{\alpha}_t(x_t) \\ \text{subject to} \quad & A_t x_t \geq b_t - E_{t-1} \hat{x}_{t-1}. \end{aligned} \quad (15)$$

Store the optimal solution as $\hat{x}_t$.

Step (e). Calculate the upper bound

$$\bar{z} = \sum_{t=1}^T c_t \hat{x}_t. \quad (16)$$

Step (f). Repeat for $t = T, T-1, \dots, 2$ (*backward recursion*).

Solve the optimization problem for stage t , trial decision $\hat{x}_{t-1}$:

$$\begin{aligned} \text{Min} \quad & c_t x_t + \hat{\alpha}_t(x_t) \\ \text{subject to} \quad & A_t x_t \geq b_t - E_{t-1} \hat{x}_{t-1}. \end{aligned} \quad (17)$$

Let π_{t-1} be the multiplier associated to the constraints of problem (17) at the optimal solution; use this multiplier to construct an additional supporting hyperplane for the approximate future cost function in the *previous* stage, $\hat{\alpha}_{t-1}(x_{t-1})$.

Step (g). Go to Step (b).

Problem (10)

$$z = \min_{x_t, \alpha} c_t x_t + \hat{\alpha}_t(x_t)$$

subject to

$$A_t x_t \geq b_t$$

Forward simulation: calculate optimal decision variables, i.e., $\hat{x}_t$

Backward recursion: calculate the dual multipliers, i.e., π_t to construct cuts or hyperplane for approximate future cost function

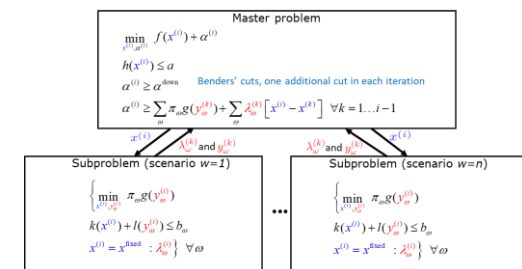
Stochastic Dual dynamic programming (SDDP)

- The DDP approach described can also be extended to the stochastic case using two-stage stochastic problem
- In the first stage, a decision x_1 is taken/given there will be m second stage subproblems:

$$\alpha_{1j}(x_1) = \min_{x_{2j}} c_2 x_{2j}$$

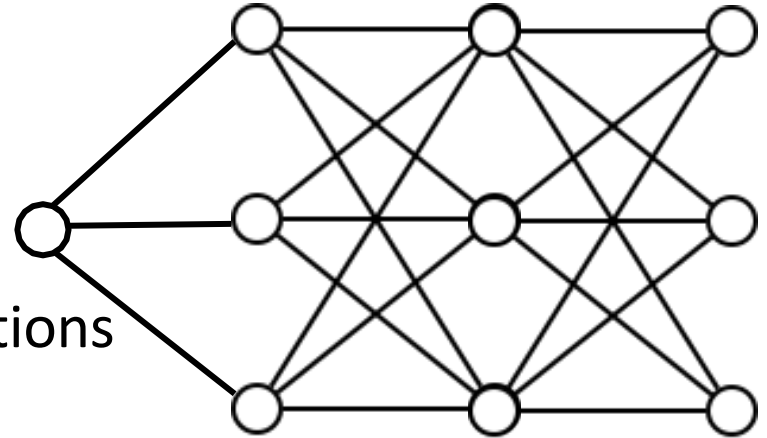
subject to $A_2 x_{2j} \leq b_{2j} - E_1 x_1 \quad j = 1, \dots, m$

- The objective is $z = \min (c_1 x_1 + \sum_{j=1}^m p_j c_2 x_{2j})$, where p_j represents the probability of each subproblem ($\sum_{j=1}^m p_j = 1$).



Stochastic Dual dynamic programming (SDDP)

- “Recombining” scenario tree
- Stage t has N_t independent realizations
- One expected cost-to-go function per stage

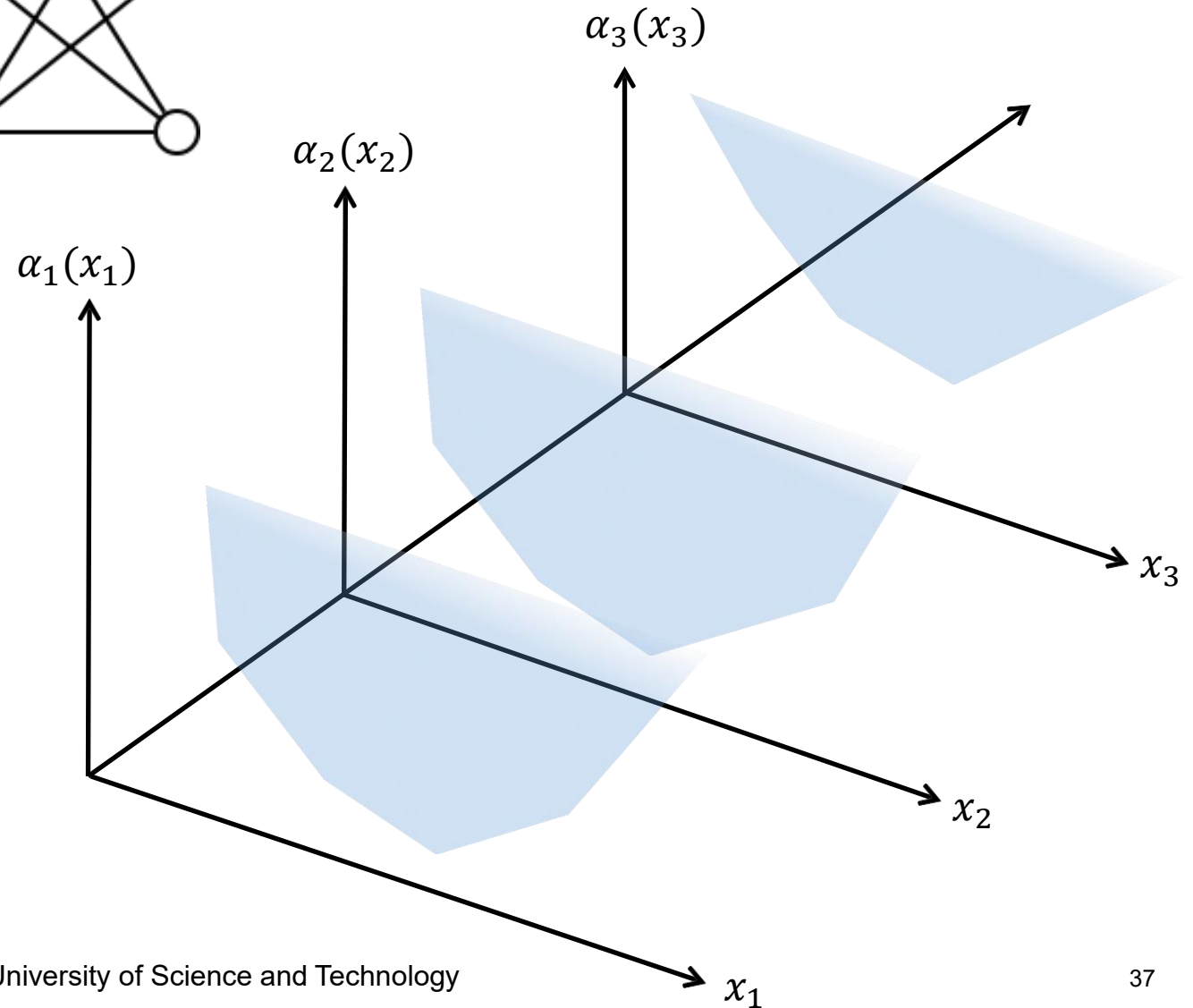
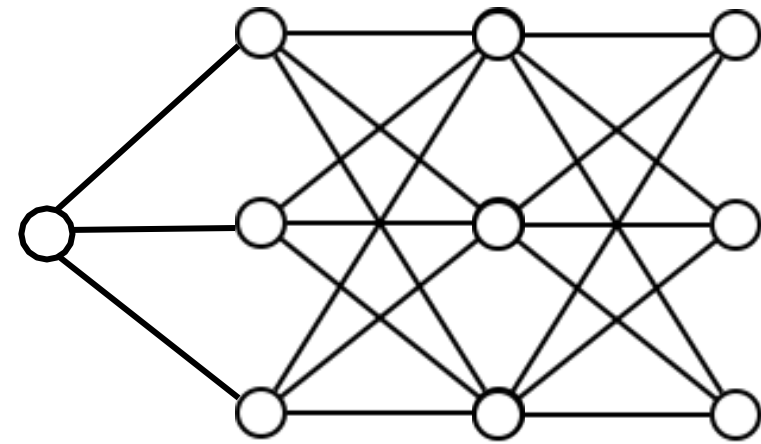


$$\alpha_n(\cdot) \equiv \alpha_t(\cdot) \quad \forall n \in S_t$$

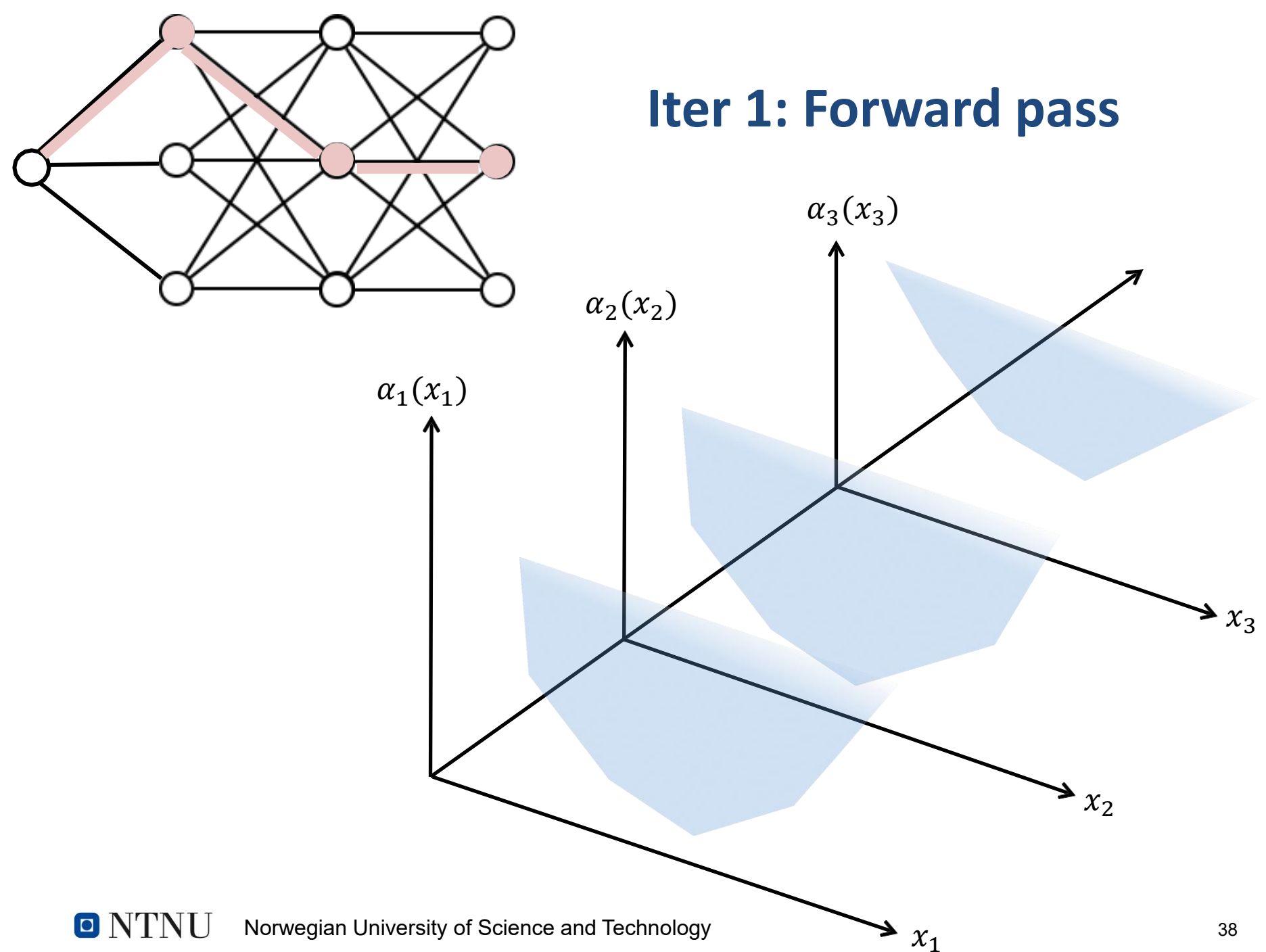
- $\alpha_t(\cdot)$ implicitly encodes solution (policy): action under j -th realization in stage t given previous action x_{t-1} is

$$(x_t, y_t) = \arg \min_{x, y} \left\{ f_{j,t}(x, y) + \alpha_t(x) : (x_{t-1}, x, y) \in X_{j,t} \right\}$$

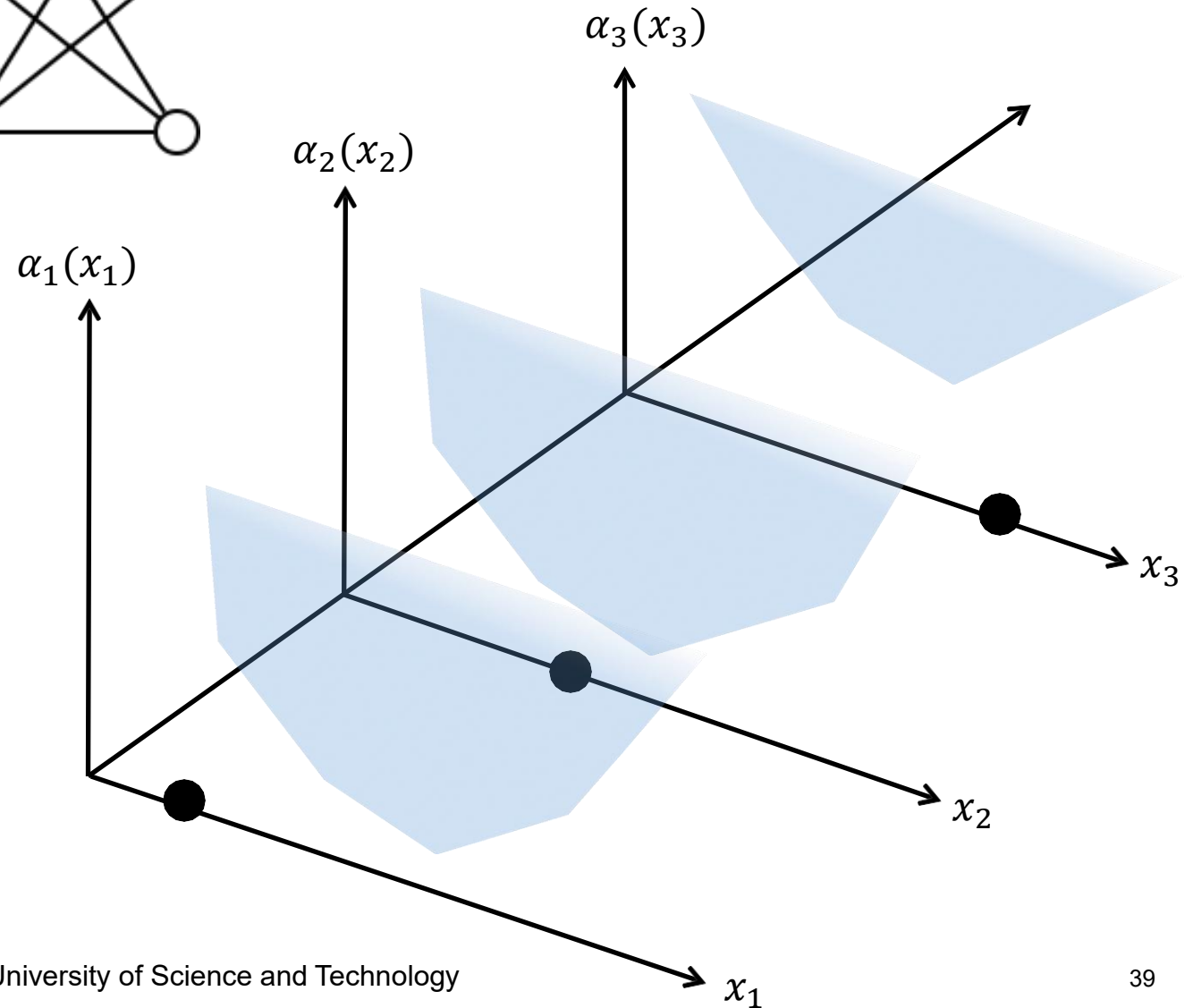
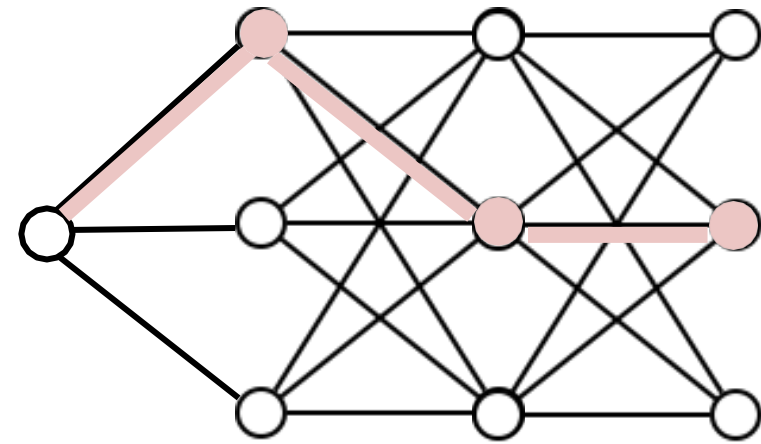
Illustration of SDDP



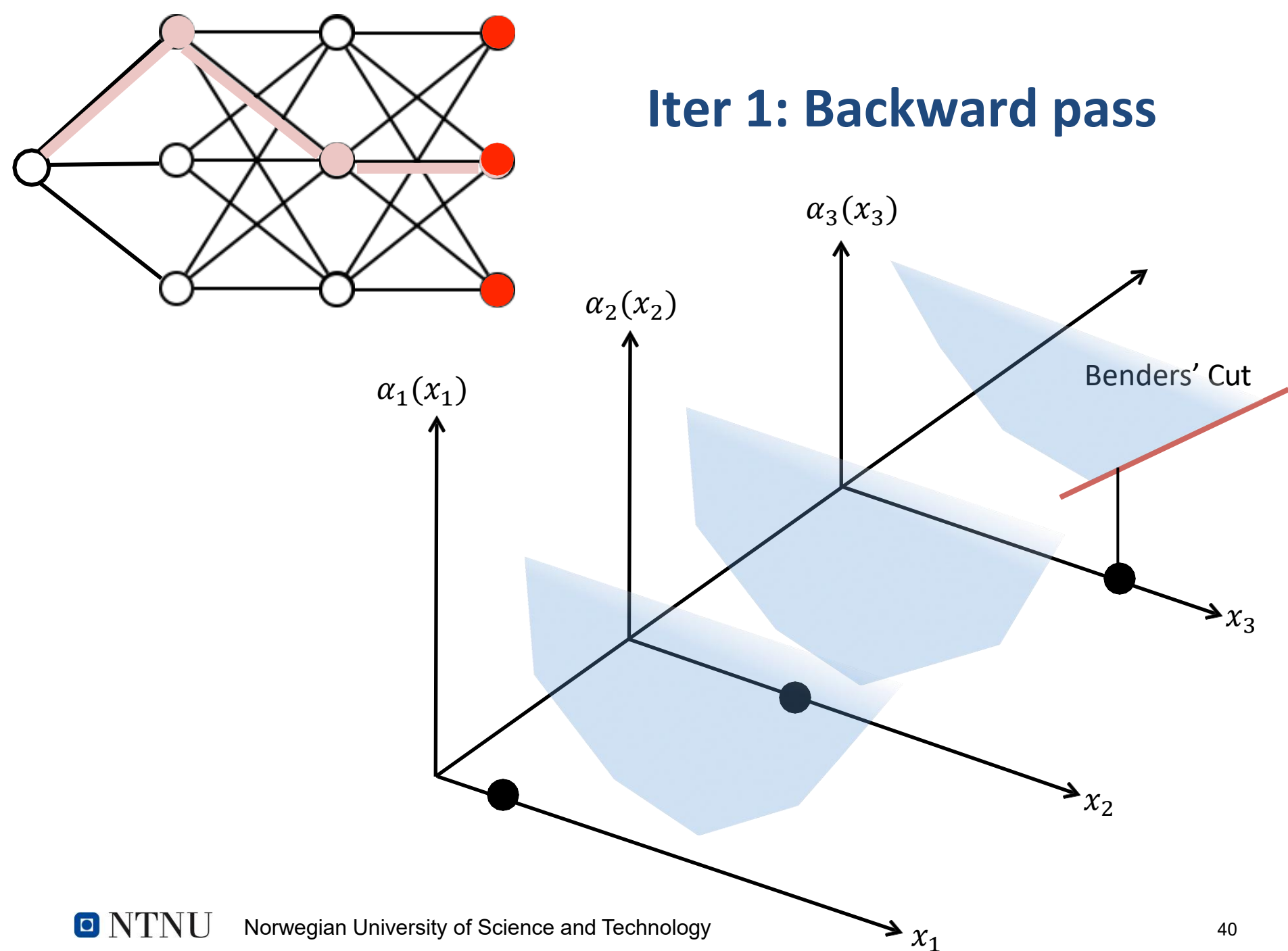
Iter 1: Forward pass



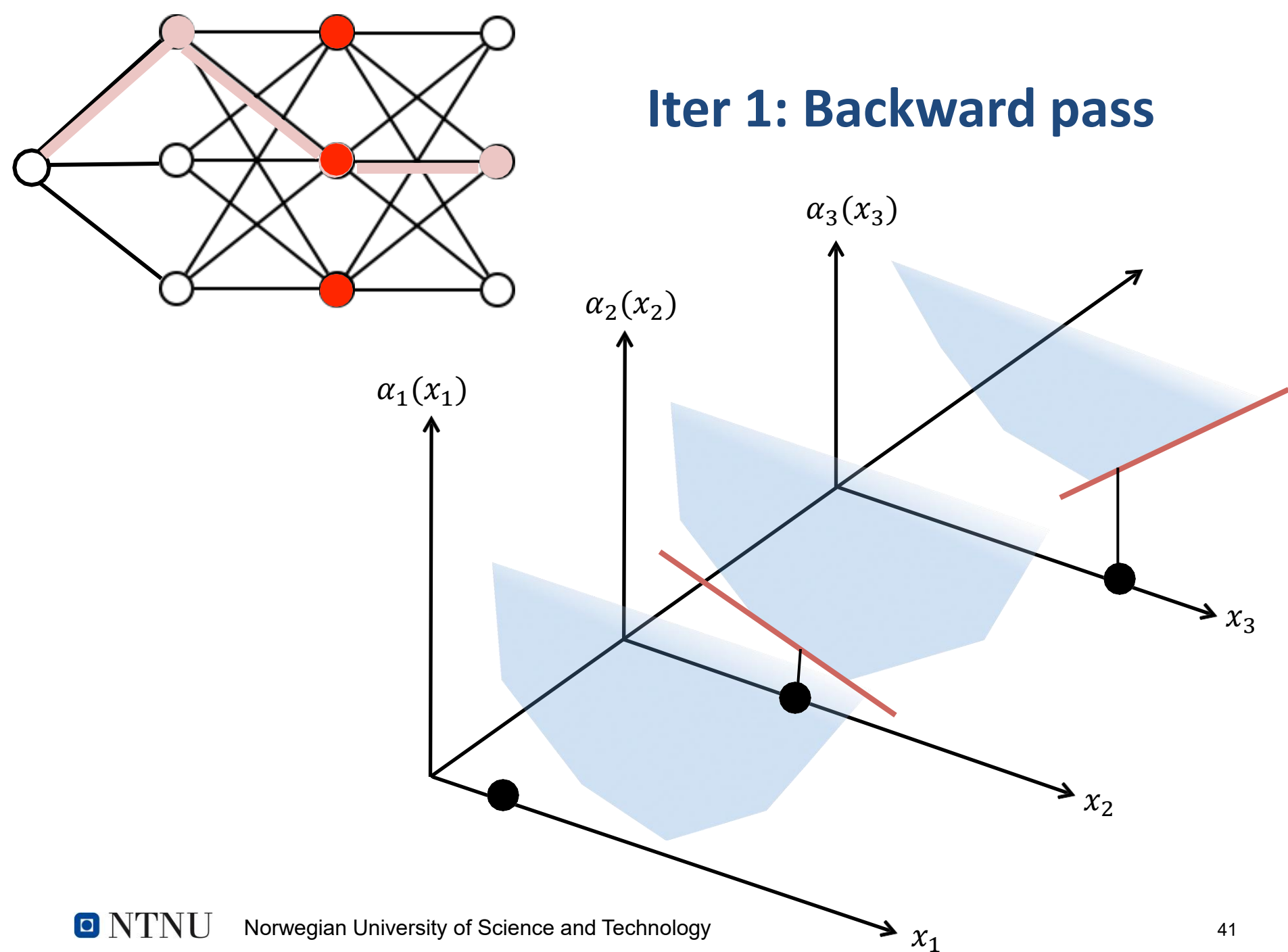
Iter 1: Forward pass



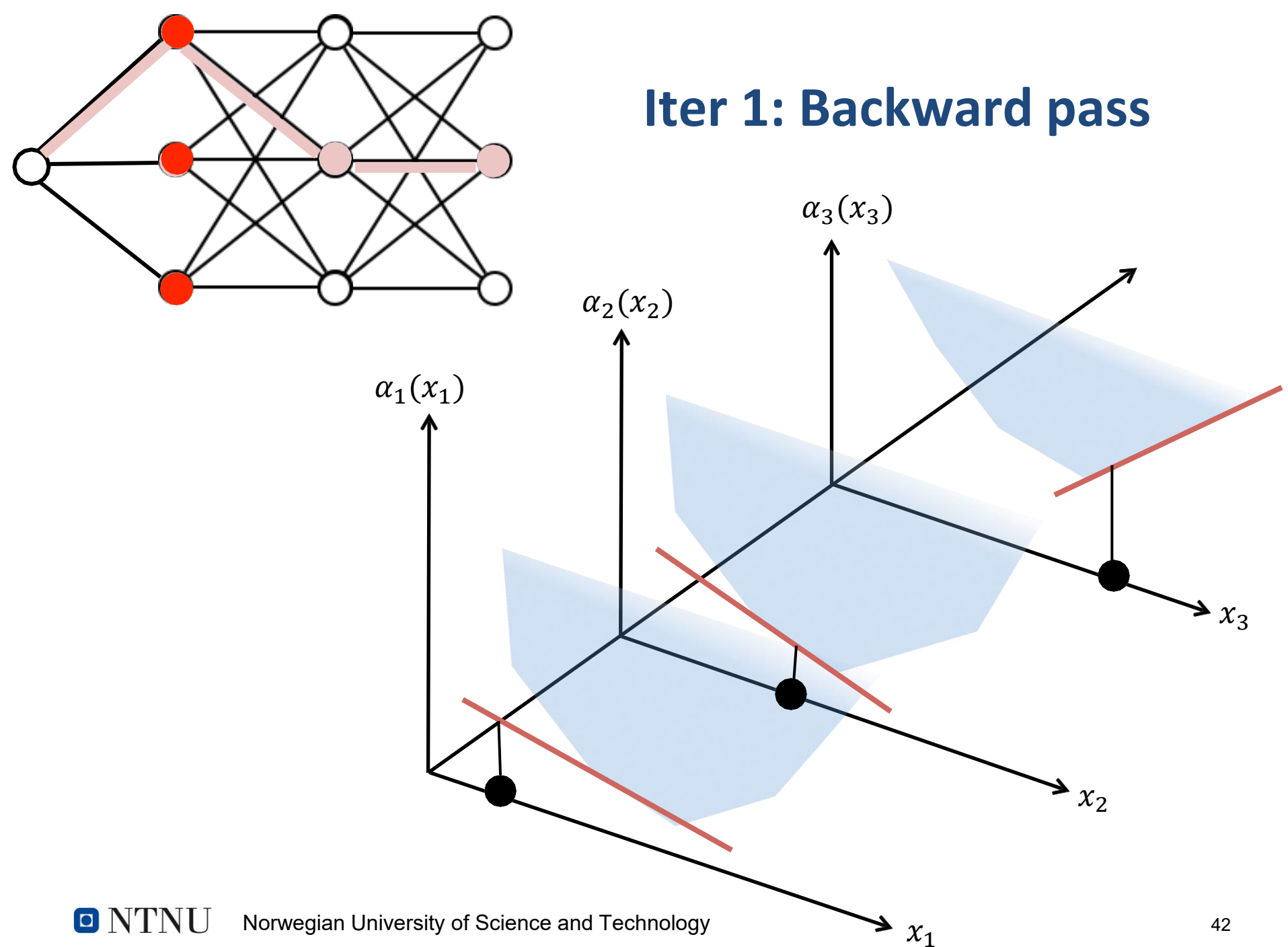
Iter 1: Backward pass



Iter 1: Backward pass

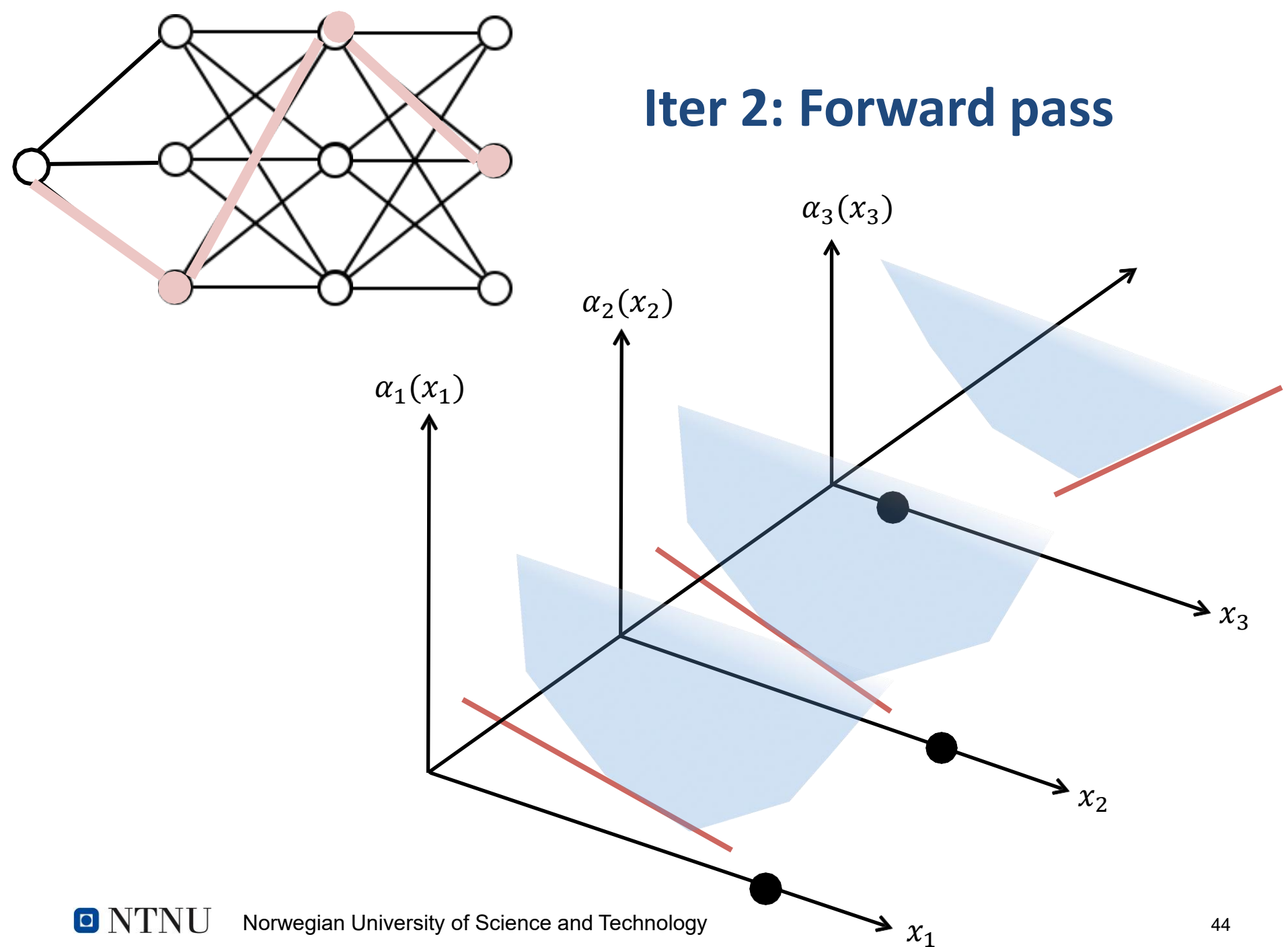


Iter 1: Backward pass

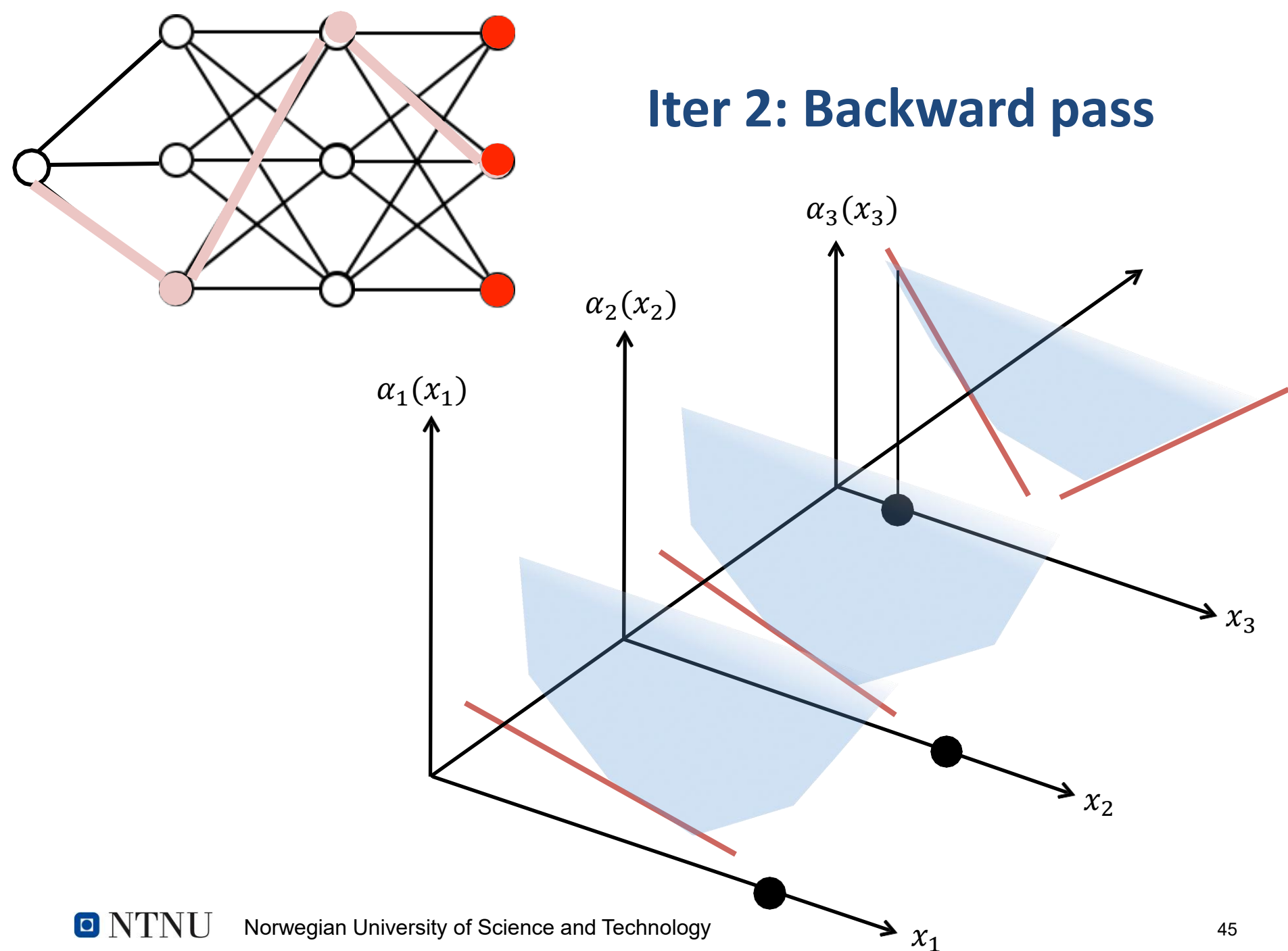




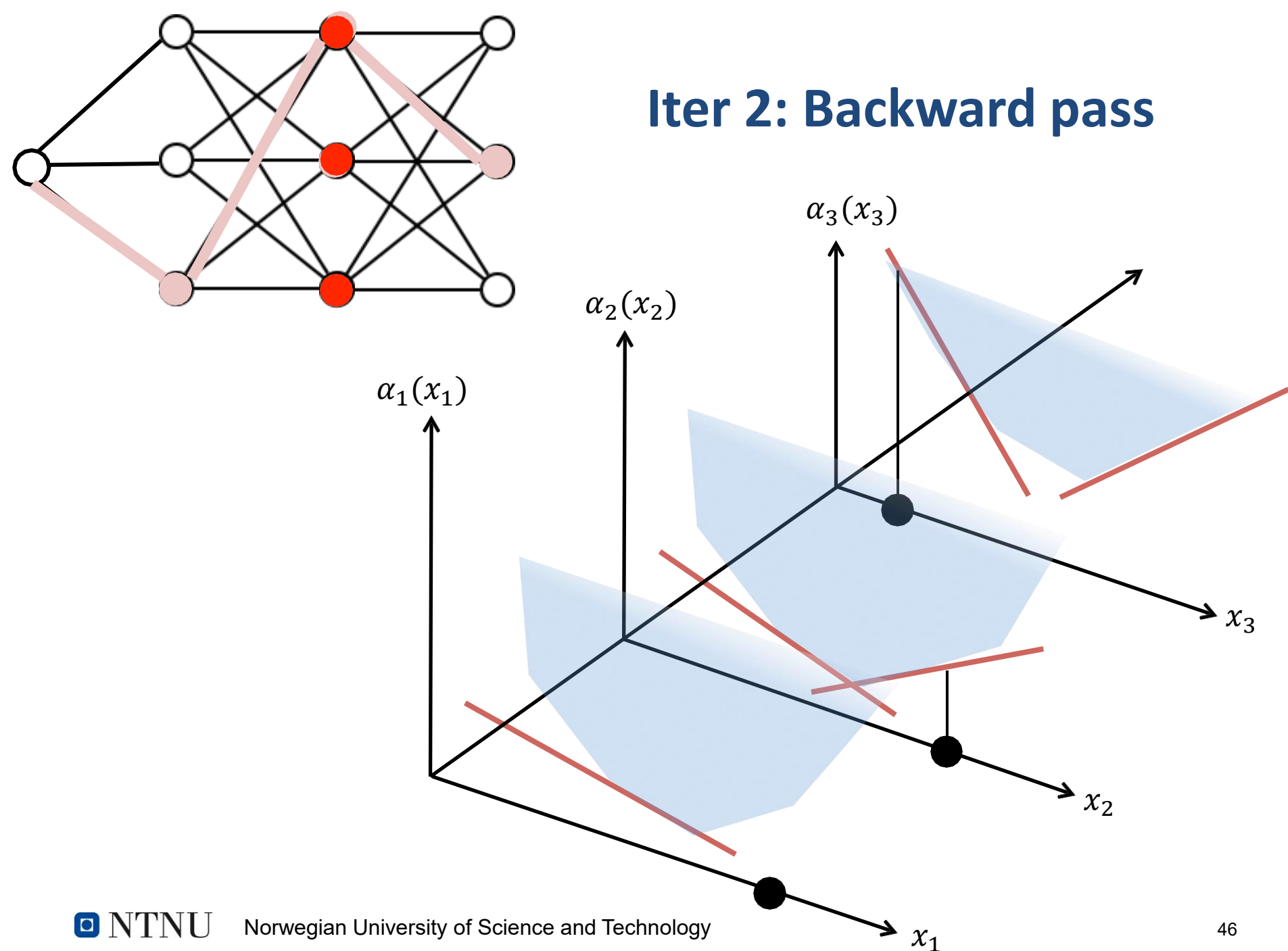
Iter 2: Forward pass



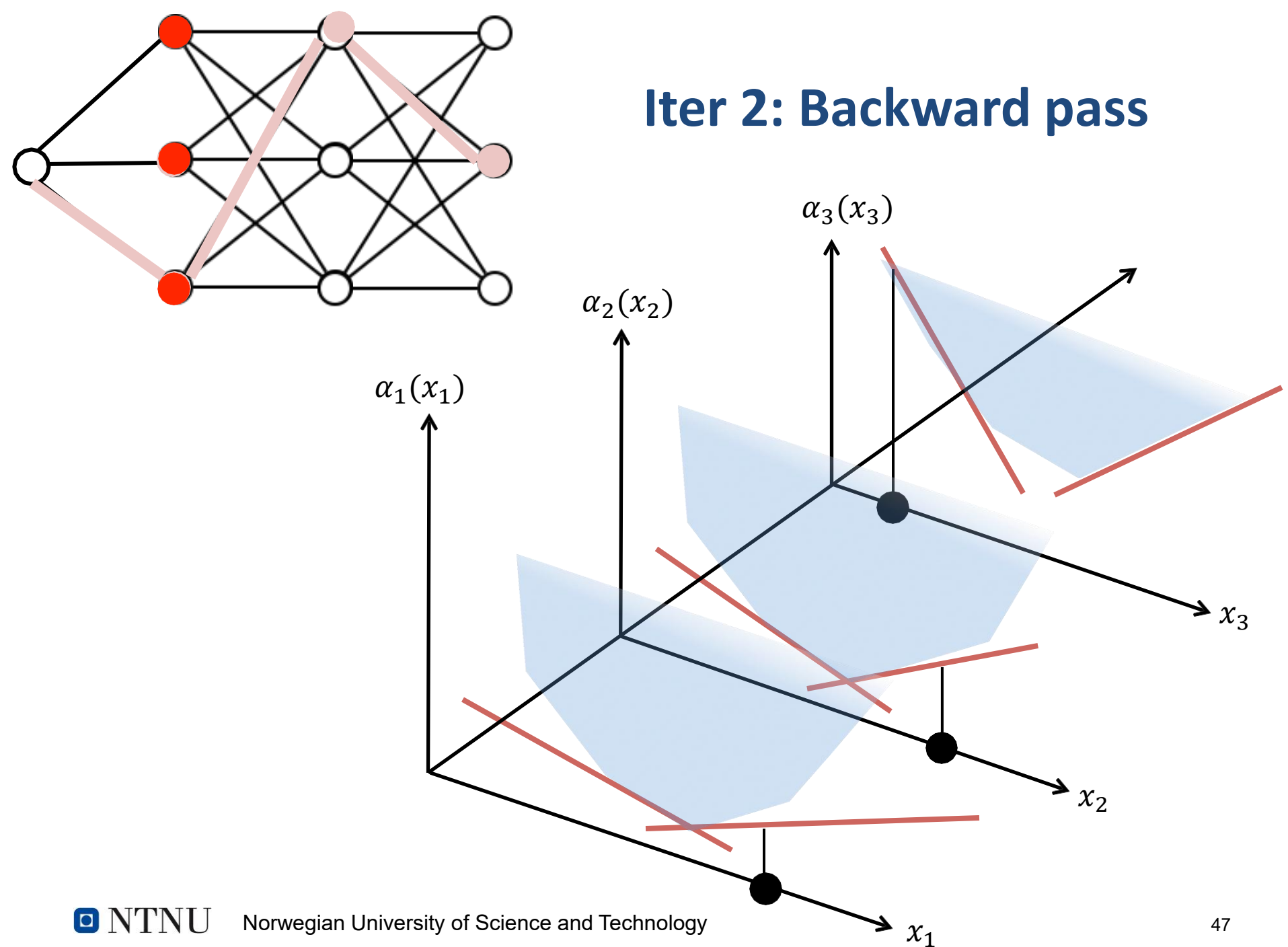
Iter 2: Backward pass



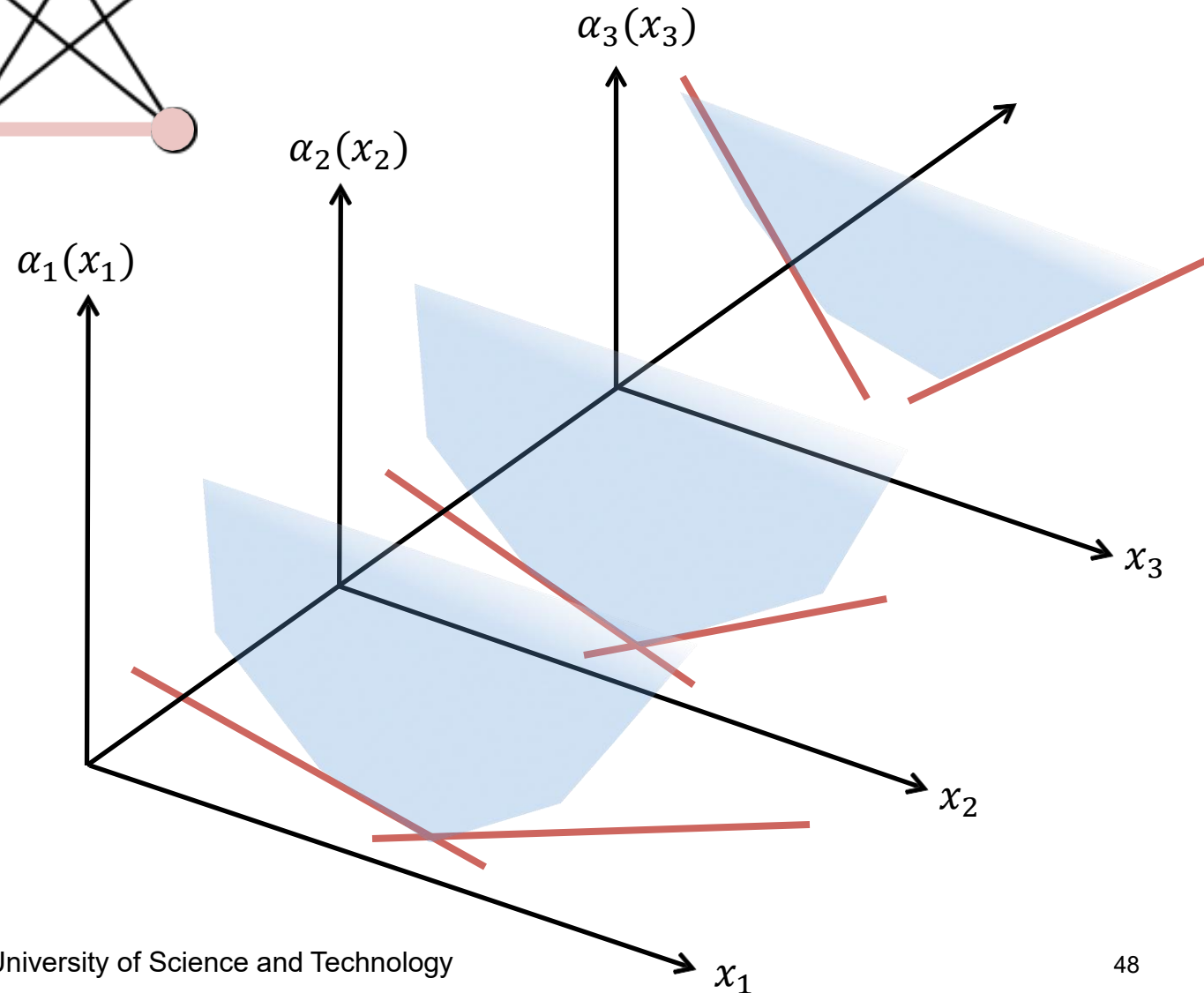
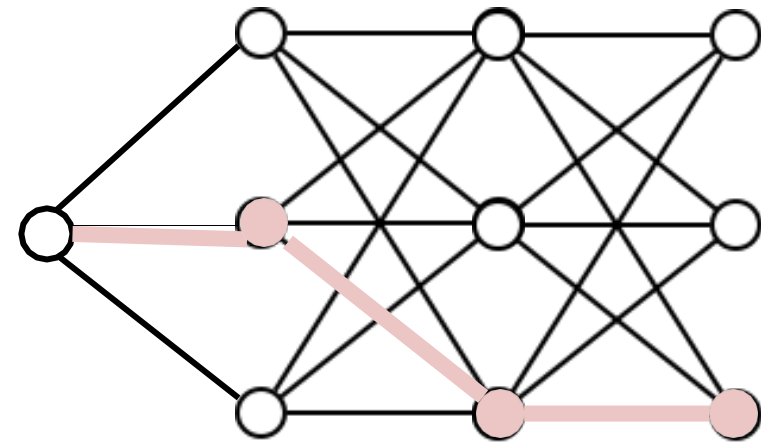
Iter 2: Backward pass



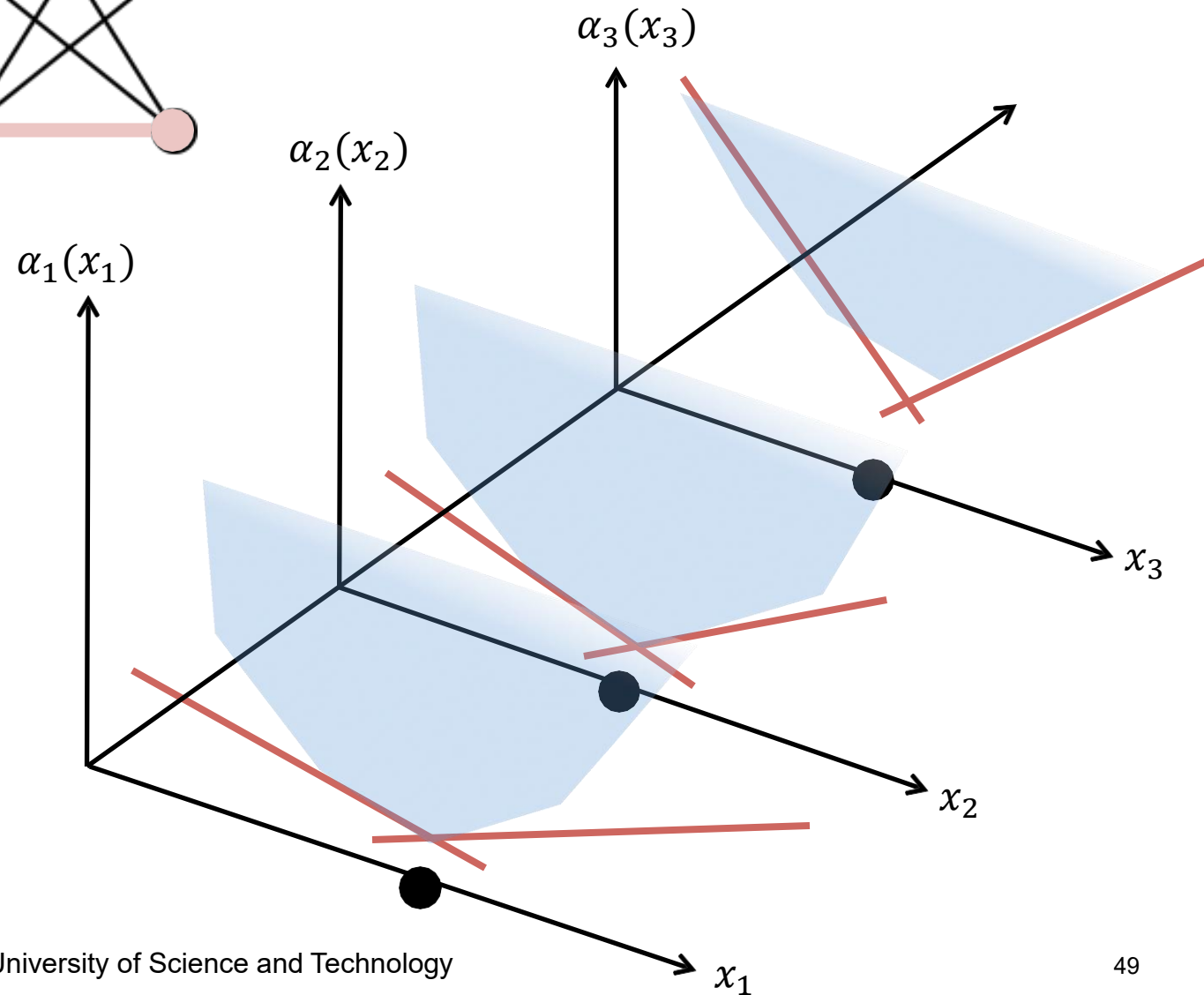
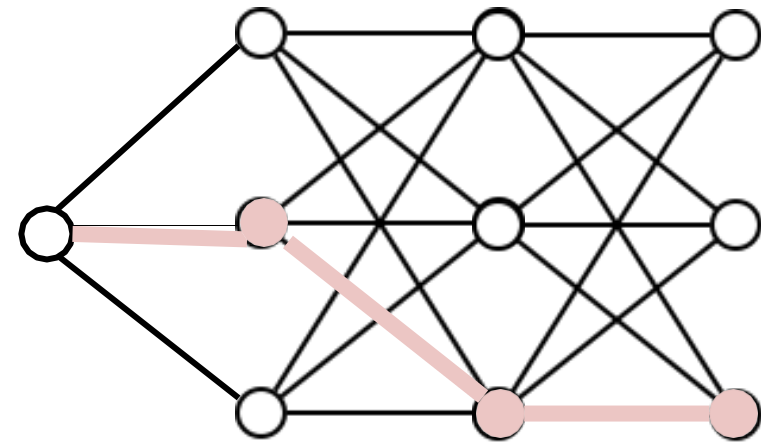
Iter 2: Backward pass



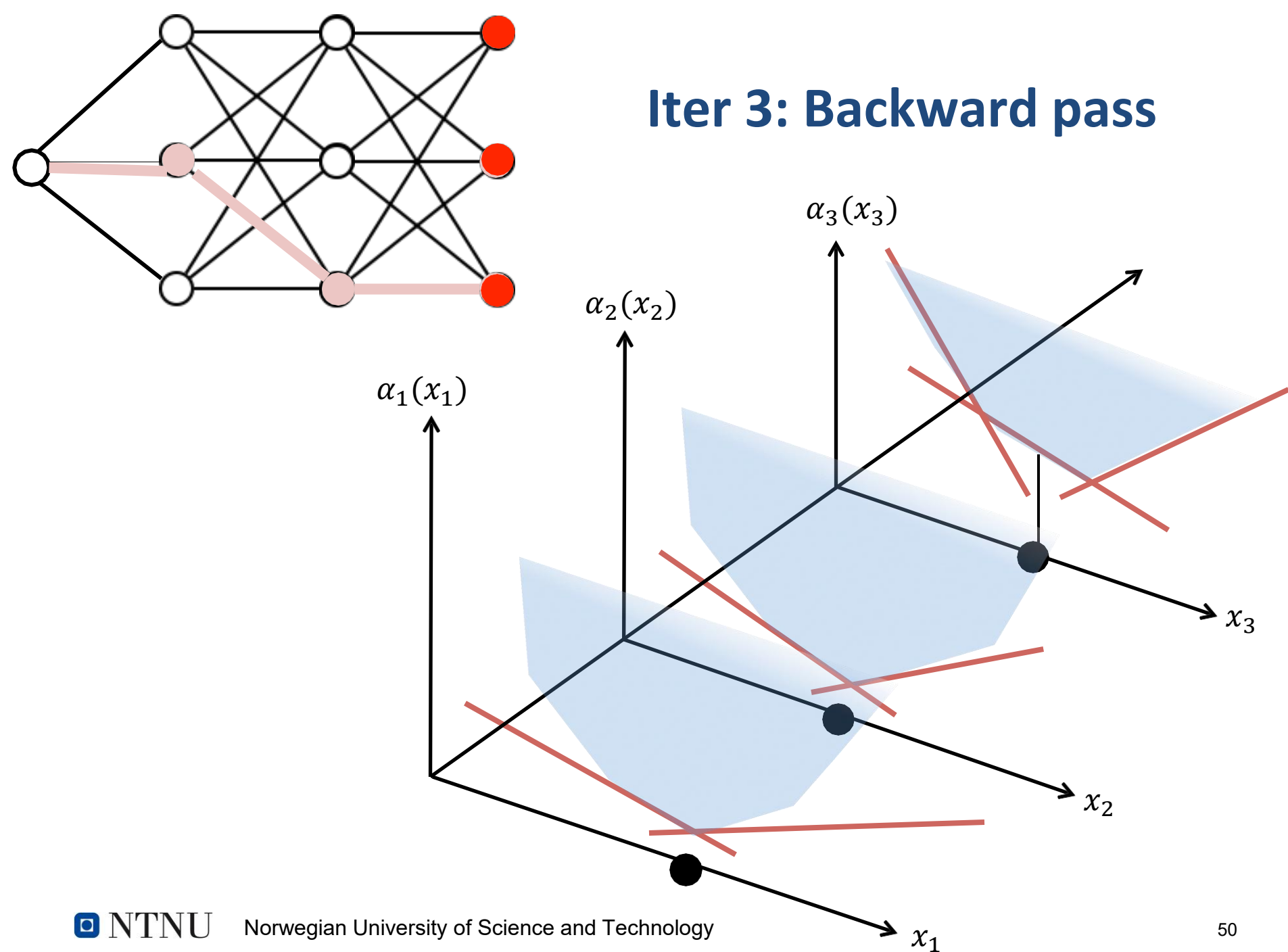
Iter 3: Forward pass



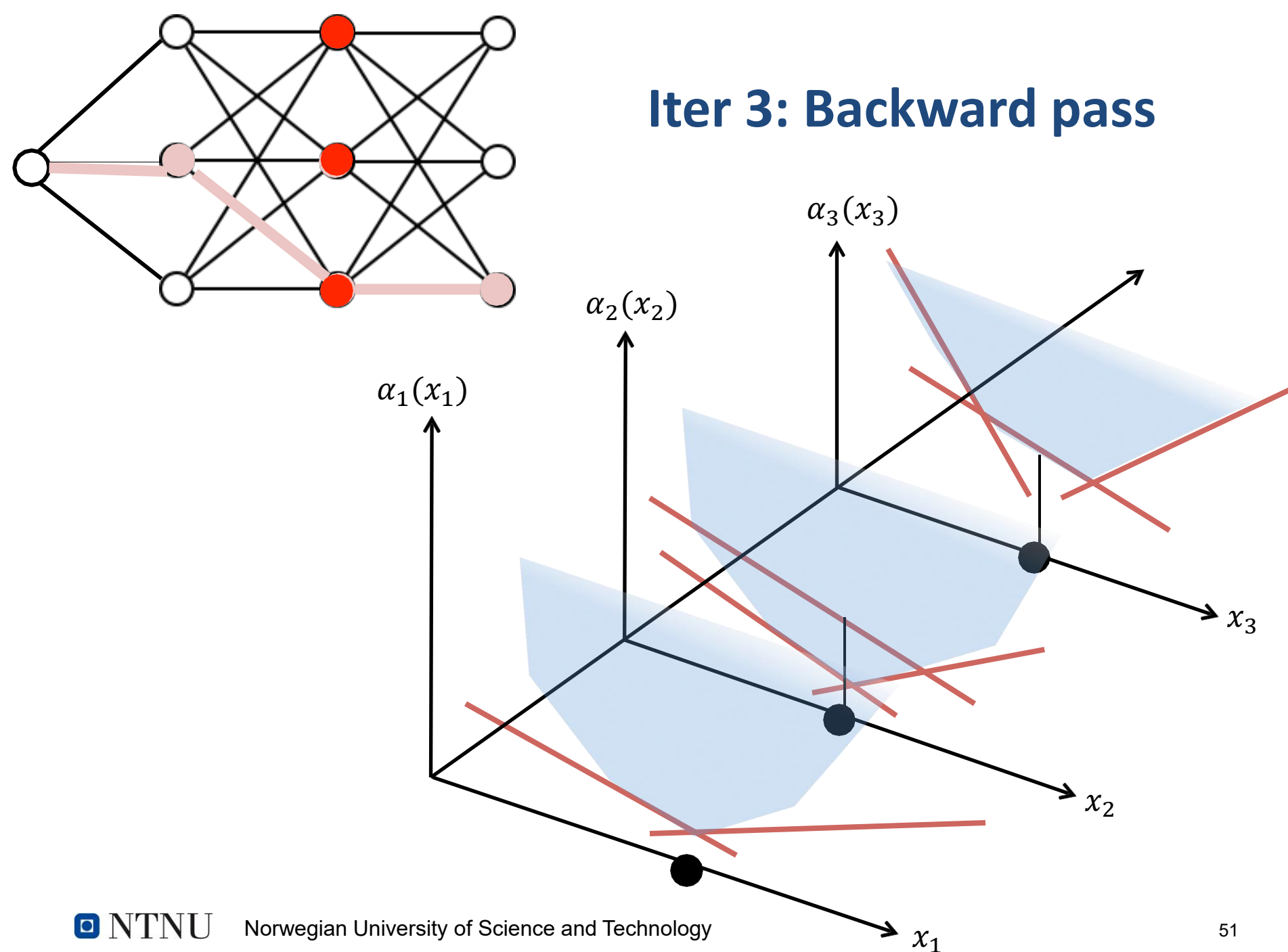
Iter 3: Forward pass



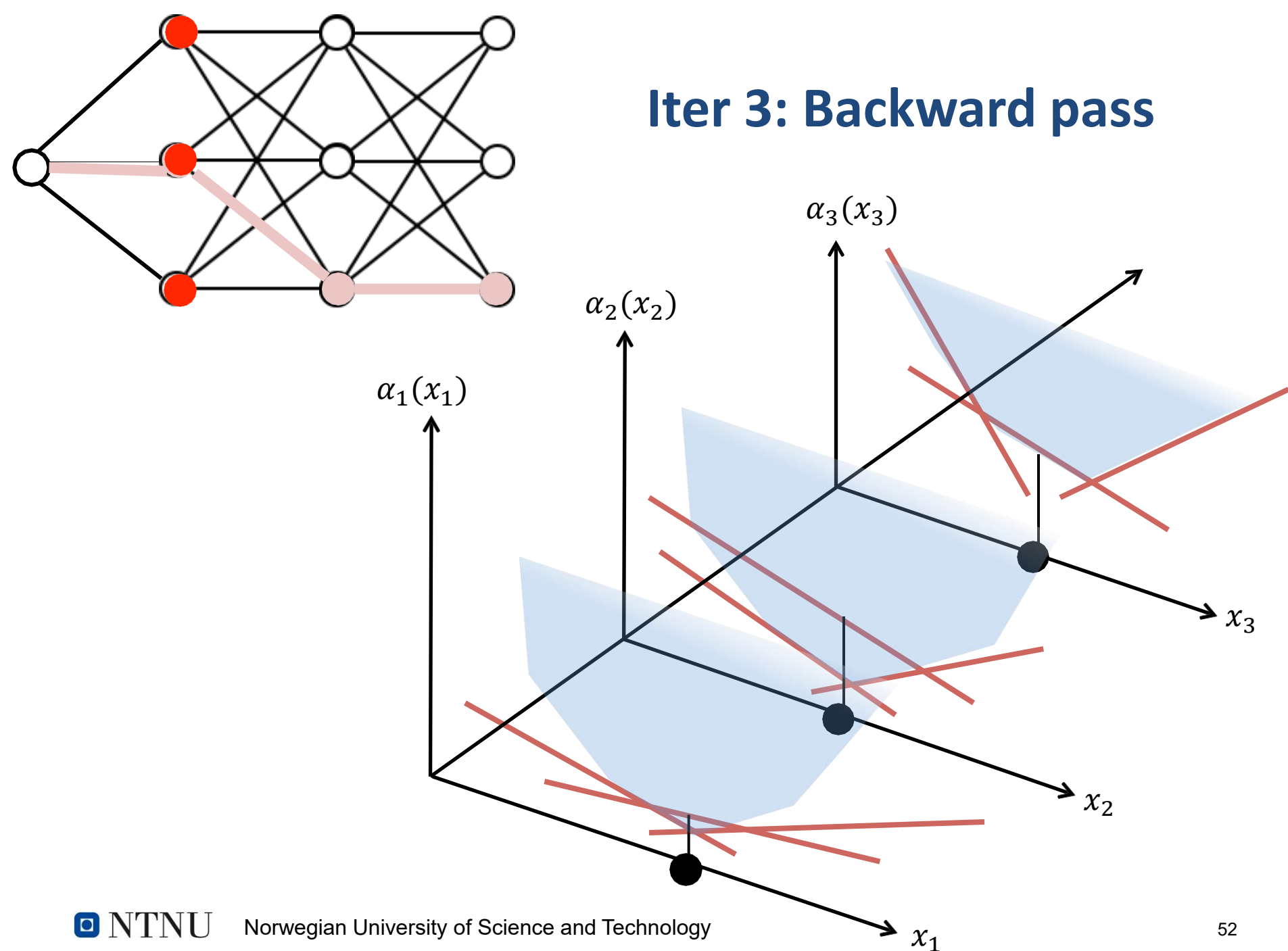
Iter 3: Backward pass



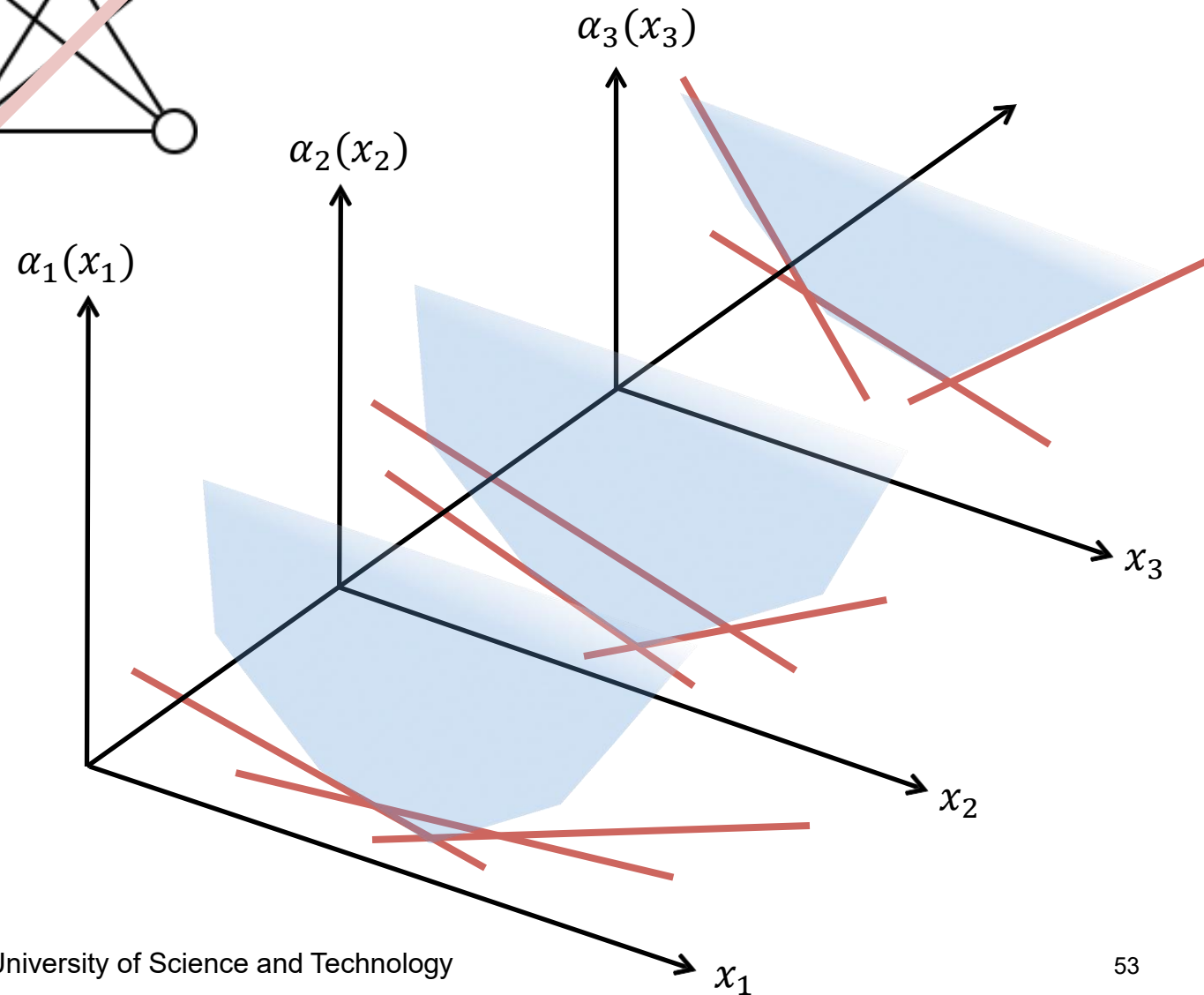
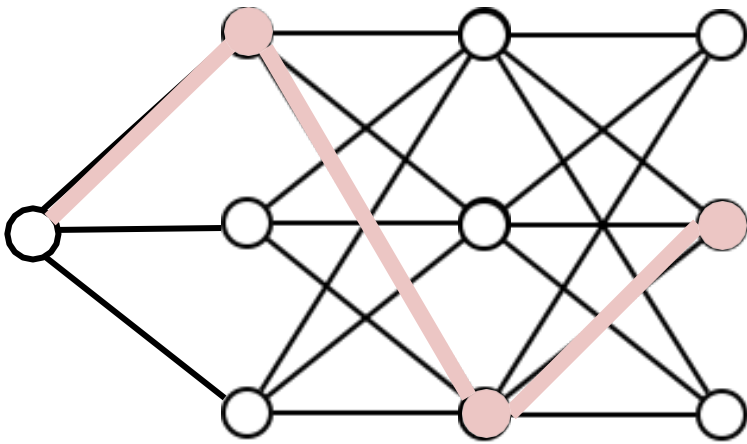
Iter 3: Backward pass



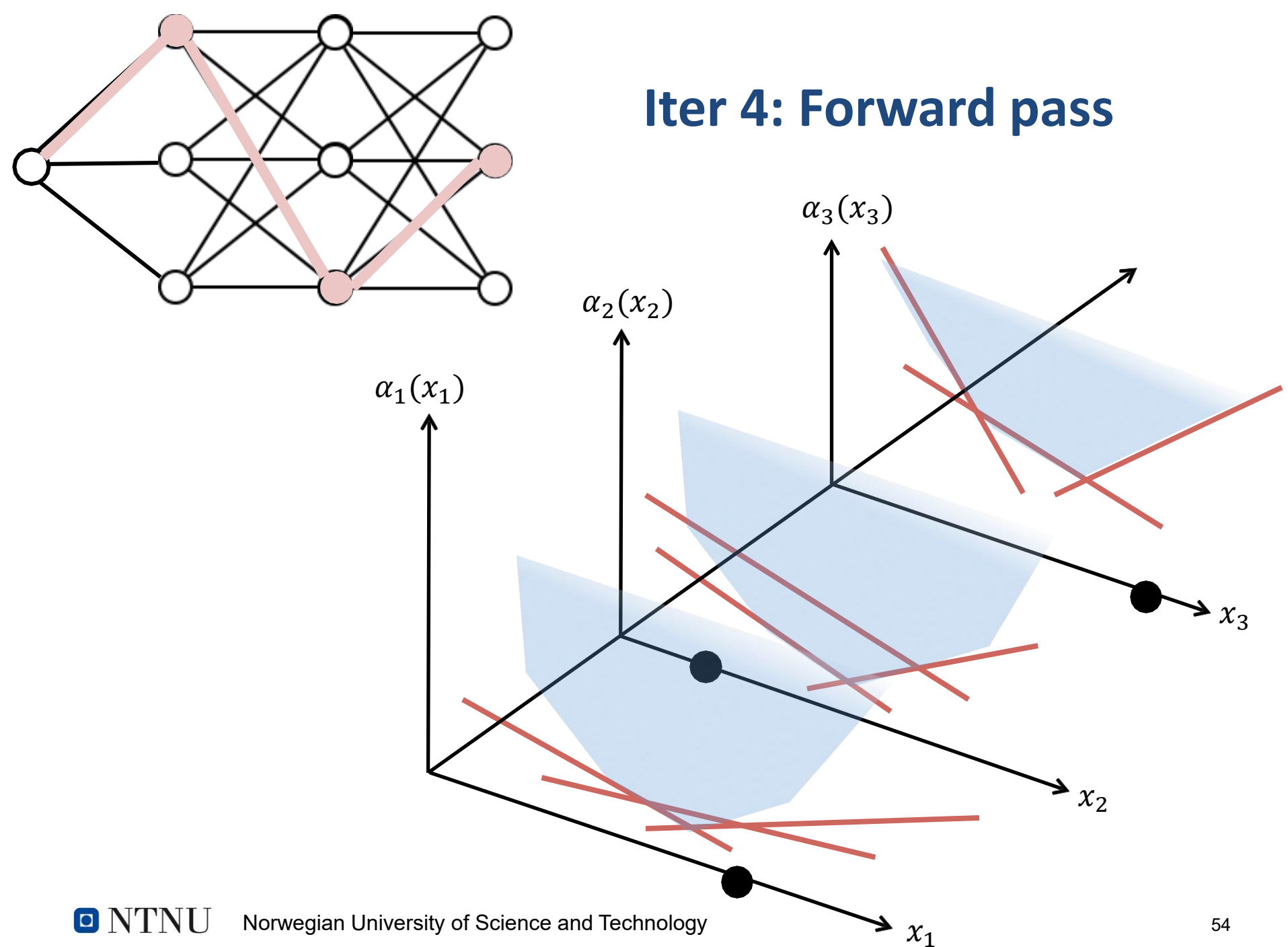
Iter 3: Backward pass



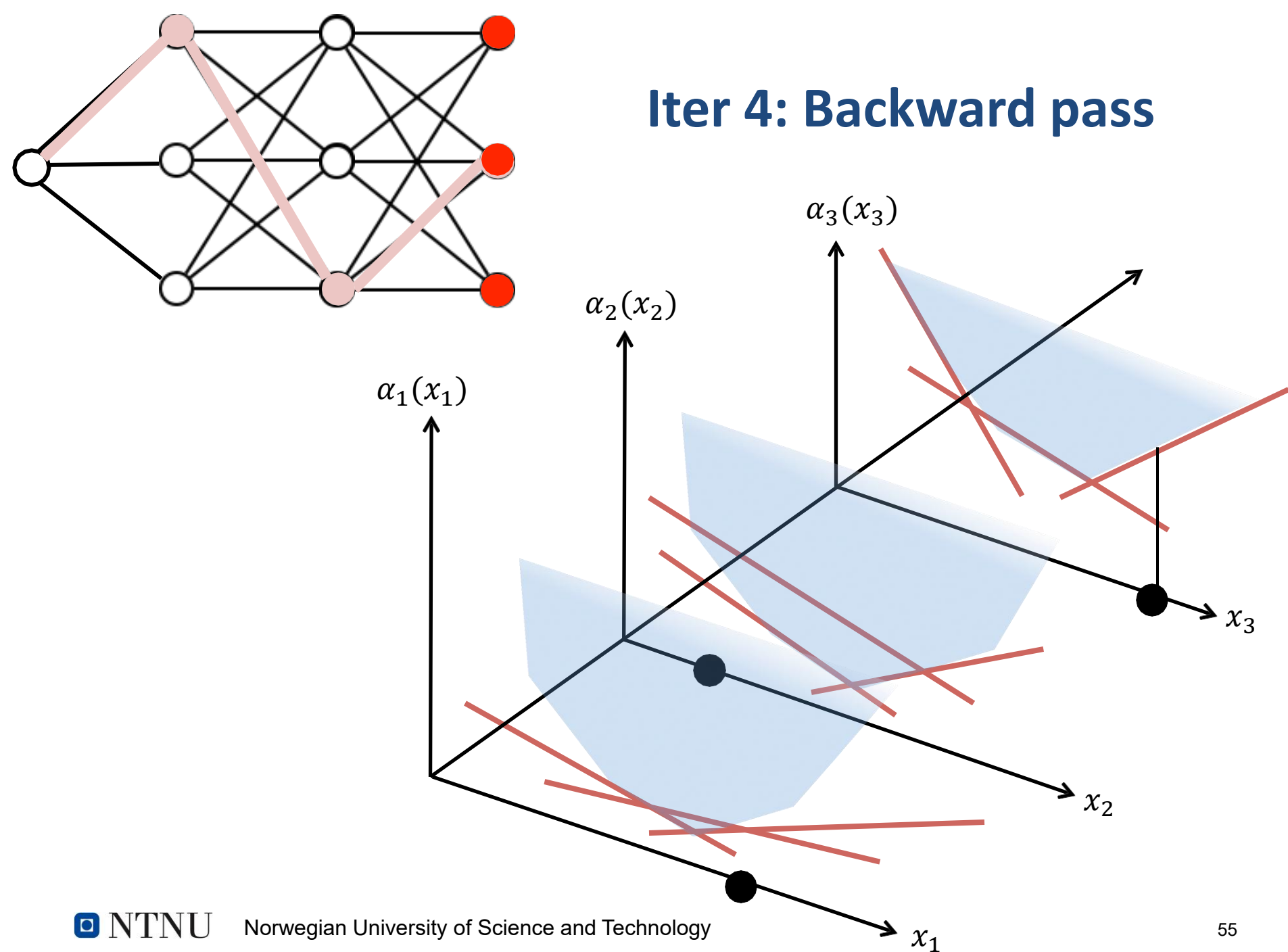
Iter 4: Forward pass



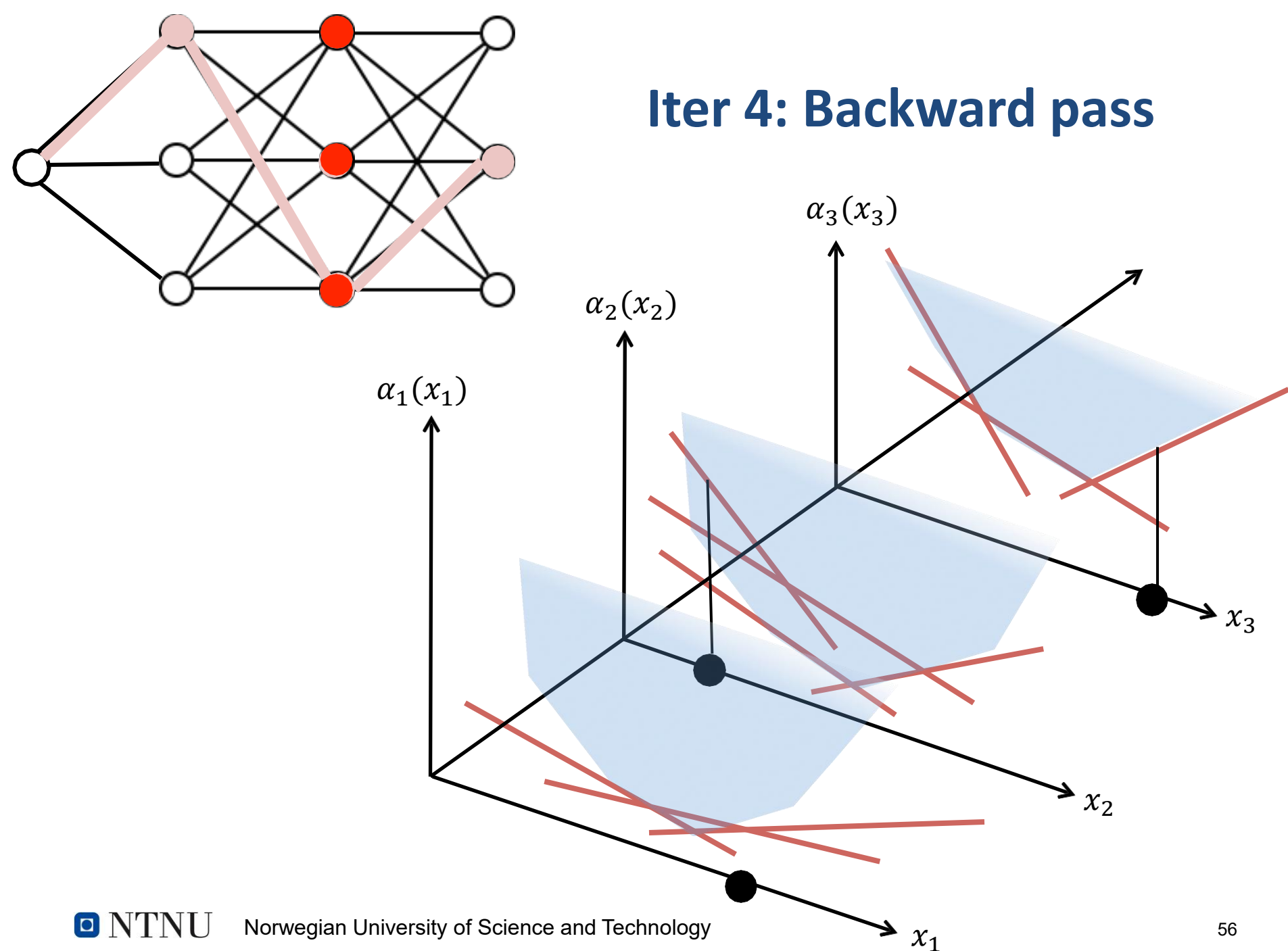
Iter 4: Forward pass



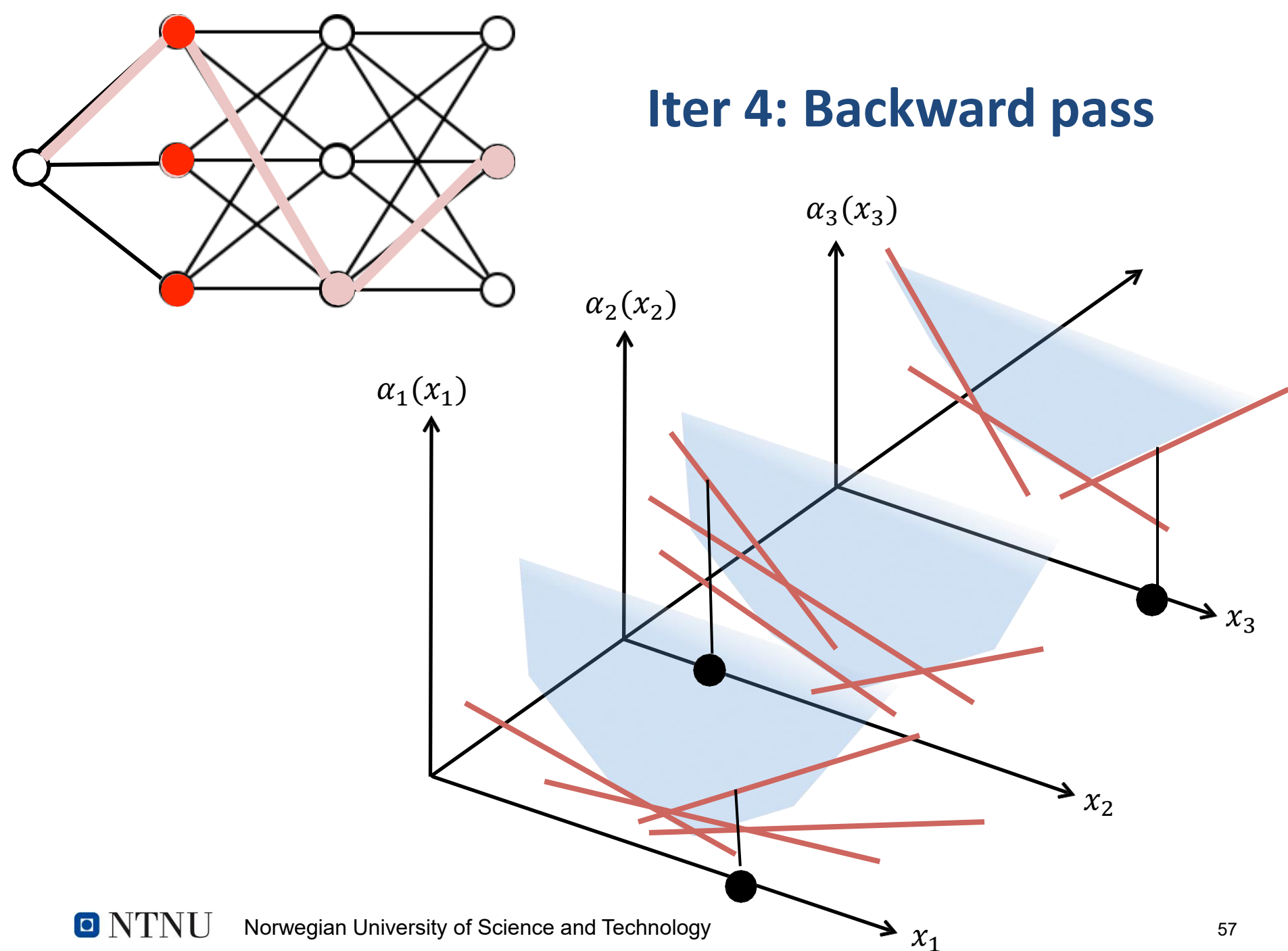
Iter 4: Backward pass



Iter 4: Backward pass



Iter 4: Backward pass



Details - Forward simulation

Optimisation problem at node (j,t) iteration i :

- $f_{j,t}(x_t, y_t)$: the immediate cost at node (j,t)
- θ_t : The expected future cost, approximated by cuts from previous iterations

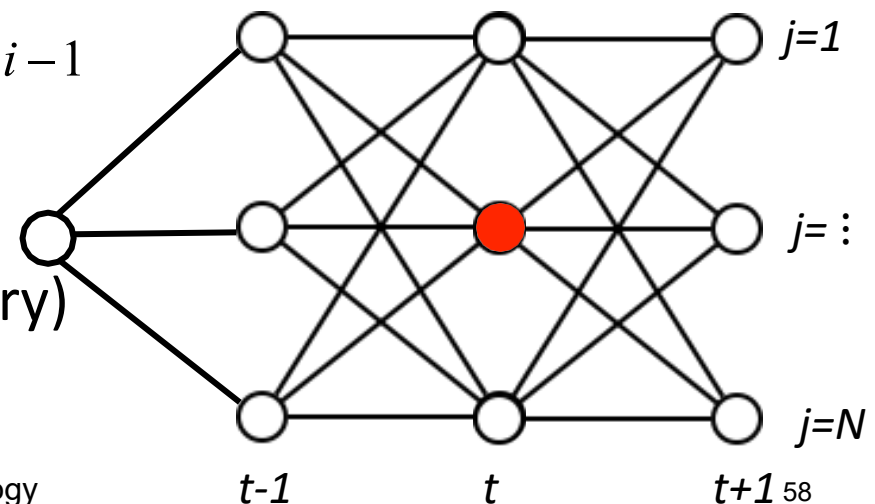
$$\underline{Q}_{j,t}^i(x_{t-1}^i) = \min_{x,y,\theta} f_{j,t}(x_t, y_t) + \theta_t$$

s.t.

$$B_{j,t}x_t + C_{j,t}y_t \geq b_{j,t} - A_{j,t}x_{t-1}^i \quad (\rho_{j,t})$$

$$\theta_t \geq \frac{1}{N} \left[\sum_{j'} v_{j',t+1}^l + \left(\pi_{j',t+1}^l \right)^T x \right] \quad \forall l = 1, \dots, i-1$$

Solve to get $\{x_t^i\}_{t=1}^T$ (sampled trajectory)



Backward recursion

Backward pass: moves in the opposite direction, from stage T to stage 1. for all j in stage $t+1$ solve:

$$\underline{Q}_{j,t+1}^{i+1}(\mathbf{x}_t^i) = \min_{x_{t+1}, y_{t+1}, \theta_{t+1}} f_{j,t+1}(x_{t+1}, y_{t+1}) + \theta_{t+1}$$

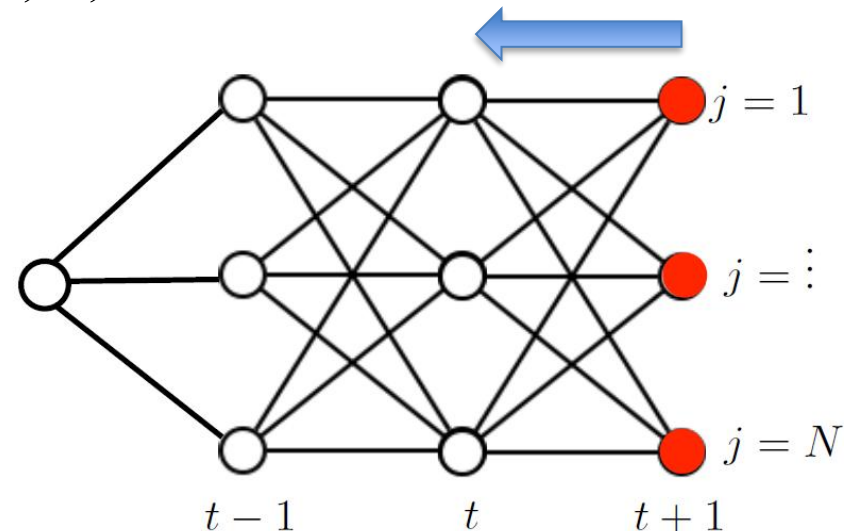
s.t.

$$B_{j,t+1}x_{t+1} + C_{j,t+1}y_{t+1} \geq b_{j,t+1} - A_{j,t+1}\mathbf{x}_t^i \quad (\rho_{j,t+1})$$

$$\theta_{t+1} \geq \frac{1}{N} \left[\sum_{j'} v_{j',t+2}^l + \left(\pi_{j',t+2}^l \right)^T x_{t+1} \right] \quad \forall l = 1, \dots, i$$

Get dual optimal solution: $\rho_{j,t+1}$

Backward pass
generates cuts from
duals and refines the
value function at stage t



Details - Backward recursion

Backward pass: moves in the opposite direction, from stage T to stage 1. for all j in stage $t+1$ solve:

$$\underline{Q}_{j,t+1}^{i+1}(\mathbf{x}_t^i) = \min_{x_{t+1}, y_{t+1}, \theta_{t+1}} f_{j,t+1}(x_{t+1}, y_{t+1}) + \theta_{t+1}$$

s.t.

$$B_{j,t+1}x_{t+1} + C_{j,t+1}y_{t+1} \geq b_{j,t+1} - A_{j,t+1}\mathbf{x}_t^i \quad (\rho_{j,t+1})$$

$$\theta_{t+1} \geq \frac{1}{N} \left[\sum_{j'} v_{j',t+2}^l + \left(\pi_{j',t+2}^l \right)^T x_{t+1} \right] \quad \forall l = 1, \dots, i$$

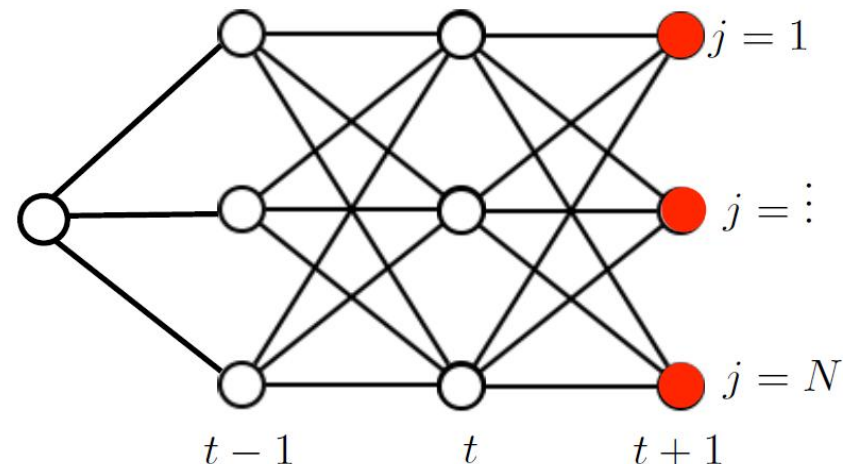
Backward pass generates cuts from duals and refines the value function at stage t



Compute cut coefficients:

$$v_{j,t+1}^i = \underline{Q}_{j,t+1}^{i+1}(\mathbf{x}_t^i) + \rho_{j,t+1}^T A_{j,t+1} \mathbf{x}_t^i$$

$$\left(\pi_{j,t+1}^i \right)^T = -\rho_{j,t+1}^T A_{j,t+1}$$



Details - Backward recursion

Backward pass: moves in the opposite direction, from stage T to stage 1. for all j in stage $t+1$ solve:

$$\underline{Q}_{j,t+1}^{i+1}(\mathbf{x}_t^i) = \min_{x,y,\theta} f_{j,t}(x,y) + \theta$$

s.t.

$$B_{j,t+1}x + C_{j,t+1}y \geq b_{j,t+1} - A_{j,t+1}\mathbf{x}_t^i \quad (\rho_{j,t+1})$$

$$\theta \geq \frac{1}{N} \left[\sum_{j'} v_{j',t+1}^l + (\pi_{j',t+1}^l)^T x \right] \quad \forall l = 1, \dots, i$$

Backward pass generates cuts from duals and refines the value function at stage t



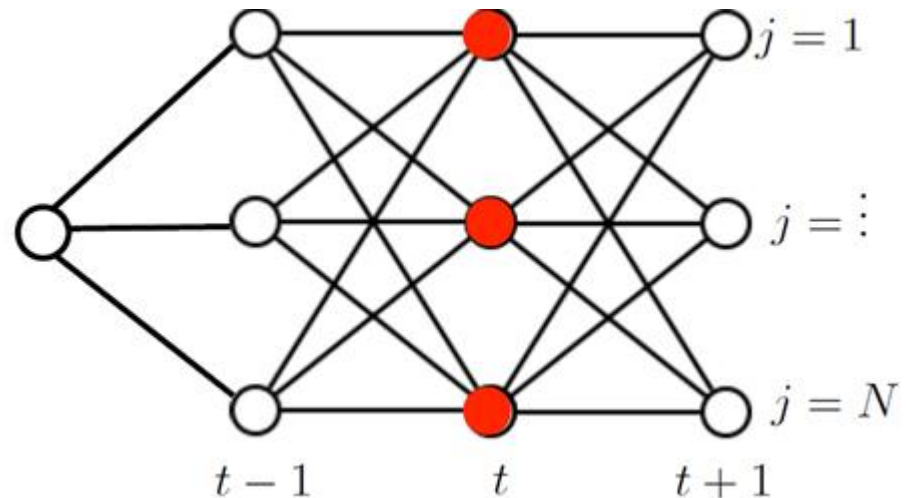
Compute cut coefficients:

$$v_{j,t+1}^i = \underline{Q}_{j,t+1}^{i+1}(\mathbf{x}_t^i) + \rho_{j,t+1}^T A_{j,t+1} \mathbf{x}_t^i$$

$$(\pi_{j,t+1}^i)^T = -\rho_{j,t+1}^T A_{j,t+1}$$

Add cut to all nodes in t :

$$\theta_{t+1}(\mathbf{x}_t) \geq \frac{1}{N} \left[\sum_{j'} v_{j',t+1}^i + (\pi_{j',t+1}^i)^T \mathbf{x}_t \right]$$



Interaction Between the Two Passes

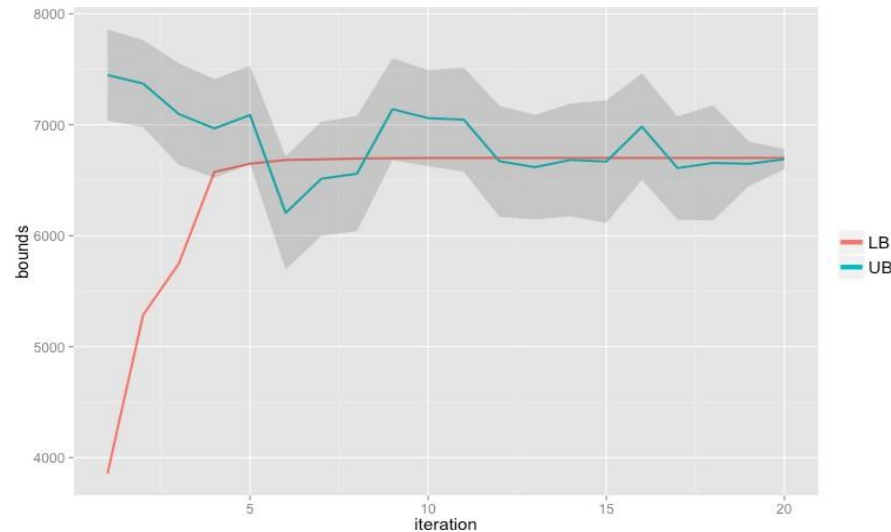
- The forward pass provides new trial states where the value function should be improved.
- The backward pass computes the subgradients of the value function at those points.
- New cuts are added, tightening the approximation.

Step	Direction	Purpose	Output
Forward sim.	$t = 1 \rightarrow T$	Simulate and obtain x_t^i	Trial states
Backward rec.	$T = T \rightarrow 1$	Compute duals, generate cuts	Cut coefficients (π, v)

- The process continues until the expected upper and lower bounds of the objective function converge within a predefined tolerance.

SDDP Bounds

- Stochastic algorithm
- Forward pass generates candidate solutions along sample paths, average of these objective values provides a statistical upper bound
- Optimal value of stage 1 problem (with lower approximation of ECTG function) provides a deterministic lower bound



Another reference

Line A. Roald, David Pozo, Anthony Papavasiliou, Daniel K. Molzahn, Jalal Kazempour, Antonio Conejo, "Power systems optimization under uncertainty: A review of methods and applications", Electric Power Systems Research, Volume 214, Part A, 2023, 108725, ISSN 0378-7796, <https://doi.org/10.1016/j.epsr.2022.108725> .

